

NeetCode Exclusives

32 questions not in Set 1 · Priority-Grouped ·

1×1 Portrait

Python Only · Priority-Grouped ·

Difficulty-First · 2×1 Landscape

32 questions · 9 rounds · Inline Quick Review ·

Chapter 2 Summary

**High Easy → High Med → High Hard → Mid Easy
→ Mid Med → Mid Hard → Low Easy → Low Med
→ Low Hard**

Contents

★ = LeetCode Premium (Premium 98 list)

Round 2 | High | M Medium (6)

Sliding Window (1)

☐ ● #567 Permutation in String

Binary Search (1)

☐ ● #875 Koko Eating Bananas

Trees (1)

☐ ● #1448 Count Good Nodes in Binary Tree

Graphs (1)

☐ ● #684 Redundant Connection

Advanced Graphs (2)

☐ ● #743 Network Delay Time

☐ ● #1584 Min Cost to Connect All Points

Round 3 | High | H Hard (3)

Intervals (1)

☐ ● #1851 Minimum Interval to Include Each Query

Advanced Graphs (2)

☐ ● #332 Reconstruct Itinerary ↗

☐ ● #778 Swim in Rising Water ↗

Round 4 | Mid | E Easy (2) ↗

Heap / Priority Queue (2) ↗

☐ ● #703 Kth Largest Element in a Stream ↗

☐ ● #1046 Last Stone Weight ↗

Round 5 | Mid | M Medium (10) ↗

Linked List (1) ↗

☐ ● #138 Copy List with Random Pointer ↗

Stack (1) ↗

☐ ● #853 Car Fleet ↗

Heap / Priority Queue (1) ↗

☐ ● #355 Design Twitter ↗

Backtracking (2) ↗

☐ ● #40 Combination Sum II ↗

☐ ● #131 Palindrome Partitioning ↗

Greedy (5) ↗

☐ ● #45 Jump Game II ↗

☐ ● #678 Valid Parenthesis String ↗

- ☐ **● #763 Partition Labels** ↗
- ☐ **● #846 Hand of Straights** ↗
- ☐ **● #1899 Merge Triplets to Form Target Triplet** ↗

Round 7 | Low | E Easy (3) ↗

1-D Dynamic Programming (1) ↗

- ☐ **● #746 Min Cost Climbing Stairs** ↗

Math & Geometry (2) ↗

- ☐ **● #66 Plus One** ↗
- ☐ **● #202 Happy Number** ↗

Round 8 | Low | M Medium (8) ↗

1-D Dynamic Programming (2) ↗

- ☐ **● #213 House Robber II** ↗
- ☐ **● #647 Palindromic Substrings** ↗

2-D Dynamic Programming (3) ↗

- ☐ **● #97 Interleaving String** ↗
- ☐ **● #494 Target Sum** ↗
- ☐ **● #518 Coin Change II** ↗

Bit Manipulation (1) ↗

#371 Sum of Two Integers

Math & Geometry (2)

#43 Multiply Strings

□ #2013 Detect Squares

Round 2 · High · Medium

Round 2

High Priority · Medium

6 questions · 5 patterns

- ☐ ● Sliding Window #567 ↗
- ☐ ● Binary Search #875 ↗
- ☐ ● Trees #1448 ↗
- ☐ ● Graphs #684 ↗
- ☐ ● Advanced Graphs #743 #1584 ↗

Sliding Window

Round 2 · High · Medium · 1 question

Sliding Window

□ #567 Permutation in String

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Two Pointers · String · Sliding Window

Lists: NeetCode 150

Problem

Given two strings $s1$ and $s2$, return true if $s2$ contains a permutation of $s1$, or false otherwise. In other words, return true if one of $s1$'s permutations is the substring of $s2$.

Example 1:

```
Input: s1 = "ab", s2 = "eidbaooo"  
Output: true  
Explanation: s2 contains one permutation of s1 ("ba").
```

Example 2:

```
Input: s1 = "ab", s2 = "eidboaoo"  
Output: false
```

Constraints:

- $1 \leq s1.length, s2.length \leq 104$
- $s1$ and $s2$ consist of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def checkInclusion(self, s1: str, s2: str) -> bool:
        count = collections.Counter(s1)
        required = len(s1)

        for r, c in enumerate(s2):
            count[c] -= 1
            if count[c] >= 0:
                required -= 1
            if r >= len(s1): # The window is oversized.

                count[s2[r - len(s1)]] += 1
                if count[s2[r - len(s1)]] > 0:
                    required += 1
            if required == 0:
                return True

        return False
```

• LeetDoocs

```
class Solution:
    def checkInclusion(self, s1: str, s2: str) -> bool:
        cnt = Counter(s1)
        need = len(cnt)
        m = len(s1)
        for i, c in enumerate(s2):
            cnt[c] -= 1
            if cnt[c] == 0:
                need -= 1
            if i >= m:
                cnt[s2[i - m]] += 1
                if cnt[s2[i - m]] == 1:
                    need += 1
            if need == 0:
                return True
        return False
```

• SimplyLeet

```
# Python solution with line-by-line comments

def checkInclusion(s1: str, s2: str) -> bool:
    # Get lengths of the input strings
    len1, len2 = len(s1), len(s2)
    # Edge case: s1 cannot be a substring if longer than s2
    if len1 > len2:
        return False

    # Initialize frequency arrays for s1 and for the first window of s2

    s1_count = [0] * 26
    window_count = [0] * 26

    # Build frequency counts for s1 and the initial window in s2
    for i in range(len1):
        s1_count[ord(s1[i]) - ord('a')] += 1
        window_count[ord(s2[i]) - ord('a')] += 1

    # Check if initial window is a permutation of s1
    if s1_count == window_count:

        return True

    # Slide the window across s2, updating the counts on the fly
    for i in range(len1, len2):
        # Add the new character from the right end of the window
        window_count[ord(s2[i]) - ord('a')] += 1
        # Remove the character that is no longer in the window
        window_count[ord(s2[i - len1]) - ord('a')] -= 1

        # Check if current window matches s1's frequency distribution

        if window_count == s1_count:
            return True

    return False

# Example usage:
# print(checkInclusion("ab", "eidbaooo")) # Expected output: True
```

◆ Quick Review

Key Insights

- Use a sliding window approach to examine substrings of $s2$ of length equal to $s1$.
- Maintain frequency counts of characters for both $s1$ and the current window in $s2$.
- Compare the frequency counts to determine if a valid permutation exists.
- Increment the window by adding a new character and removing the leftmost one, keeping the operation efficient.

Space & Time

Time Complexity: $O(n)$ where n is the length of $s2$ (each character is processed a constant number of times).

Space Complexity: $O(1)$ since the frequency count arrays have a fixed size (26 for lowercase English letters).

Solution

The solution uses two fixed-length arrays (of size 26) to store character frequencies for $s1$ and the current window in $s2$. First, initialize the frequency arrays by processing $s1$ and the

first window of s2. Then, slide the window one character at a time, updating the counts by incrementing the count for the new character and decrementing the count for the character that is no longer in the window. After each move, compare both frequency arrays; if they match, s2 contains a permutation of s1. This technique avoids repeatedly computing the frequency distribution from scratch, making the algorithm efficient.

Binary Search

Round 2 · High · Medium · 1 question

Binary Search

□ #875 Koko Eating Bananas

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search
Lists: NeetCode 150

Problem

Koko loves to eat bananas. There are n piles of bananas, the i th pile has $piles[i]$ bananas. The guards have gone and will come back in h hours. Koko can decide her bananas-per-hour eating speed of k . Each hour, she chooses some pile of bananas and eats k bananas from that pile. If the pile has less than k bananas, she eats all of them instead and will not eat any more bananas during this hour.

Koko likes to eat slowly but still wants to finish eating all the bananas before the guards return. Return the minimum integer k such that she can eat all the bananas within h hours.

Example 1:

Input: $piles = [3,6,7,11]$, $h = 8$
Output: 4

Example 2:

Input: piles = [30,11,23,4,20], h = 5
Output: 30

Example 3:

Input: piles = [30,11,23,4,20], h = 6
Output: 23

Constraints:

- $1 \leq \text{piles.length} \leq 104$
- $\text{piles.length} \leq h \leq 109$
- $1 \leq \text{piles}[i] \leq 109$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minEatingSpeed(self, piles: list[int], h: int) -> int:
        def eatHours(m: int) -> bool:
            """Returns the hours to eat all the piles with speed m."""
            return sum((pile - 1) // m + 1 for pile in piles)
        l = 1
        r = max(piles)
        return bisect.bisect_left(range(l, r), True,
                                key=lambda m: eatHours(m) <= h) + l
```

• LeetDoocs

```
class Solution:
    def minEatingSpeed(self, piles: List[int], h: int) -> int:
        def check(k: int) -> bool:
            return sum((x + k - 1) // k for x in piles) <= h
        return 1 + bisect_left(range(1, max(piles) + 1), True, key=check)
```

• SimplyLeet

```
def minEatingSpeed(piles, h):
    # Helper function to determine if Koko can finish with speed k
    def canFinish(k):
        hours = 0
        for pile in piles:
            # Calculate hours by using ceiling division for each pile
            hours += (pile + k - 1) // k
        return hours <= h

    left, right = 1, max(piles) # Speed is between 1 and the max pile element

    while left < right:
        mid = (left + right) // 2 # Candidate eating speed
        if canFinish(mid):
            right = mid # Try finding a smaller valid speed
        else:
            left = mid + 1 # Increase the speed because mid is too slow
    return left

# Example usage:
piles = [3, 6, 7, 11]

h = 8
print(minEatingSpeed(piles, h)) # Expected output: 4
```

◆ Quick Review

Key Insights

- The required eating speed k lies in the range $[1, \max(\text{piles})]$.
- For a given k , one can calculate the total hours needed by summing up $\text{ceil}(\text{pile} / k)$ for each pile.
- The problem can be solved efficiently using binary search to find the minimum k that

satisfies the time constraint.

- Early termination in the hour calculation can be used if the computed hours exceed h .

Space & Time

Time Complexity: $O(n \log(\max(\text{pile})))$ where n is the number of piles and $\max(\text{pile})$ is the maximum bananas in a pile.

Space Complexity: $O(1)$ extra space beyond the input.

Solution

We use binary search on the candidate eating rate k . The search space starts at 1 (minimum possible speed) and goes up to the maximum number of bananas in any pile. For each candidate k , we simulate the number of hours Koko would take by calculating $\text{hours} += (\text{pile} + k - 1) // k$ for each pile (this formula performs the ceiling division). If the total hours are less than or equal to h , then k might be a valid solution and we try to lower it; otherwise, we increase k . The binary search continues until we converge on the minimum valid eating speed.

Trees

Round 2 · High · Medium · 1 question

Trees

□ #1448 Count Good Nodes in Binary Tree

MED

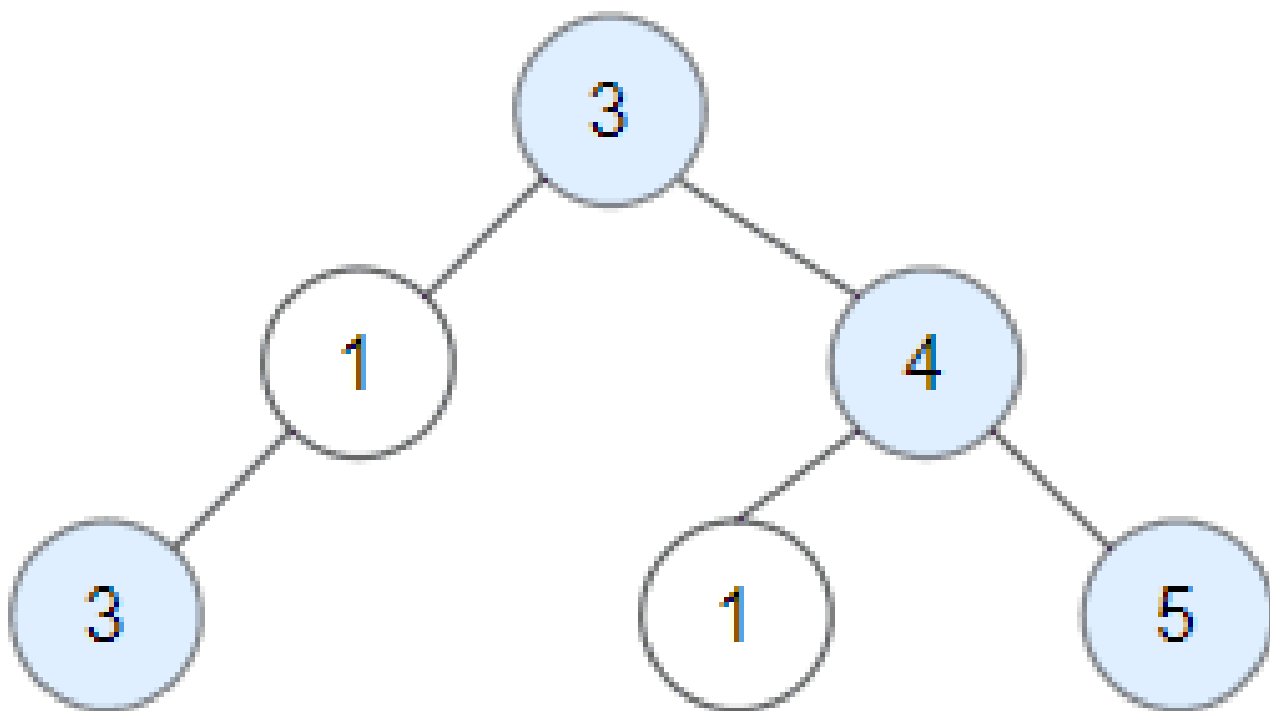
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Depth-First Search · Breadth-First
Search · Binary Tree
Lists: NeetCode 150

Problem

Given a binary tree root, a node X in the tree is named good if in the path from root to X there are no nodes with a value greater than X.

Return the number of good nodes in the binary tree.

Example 1:



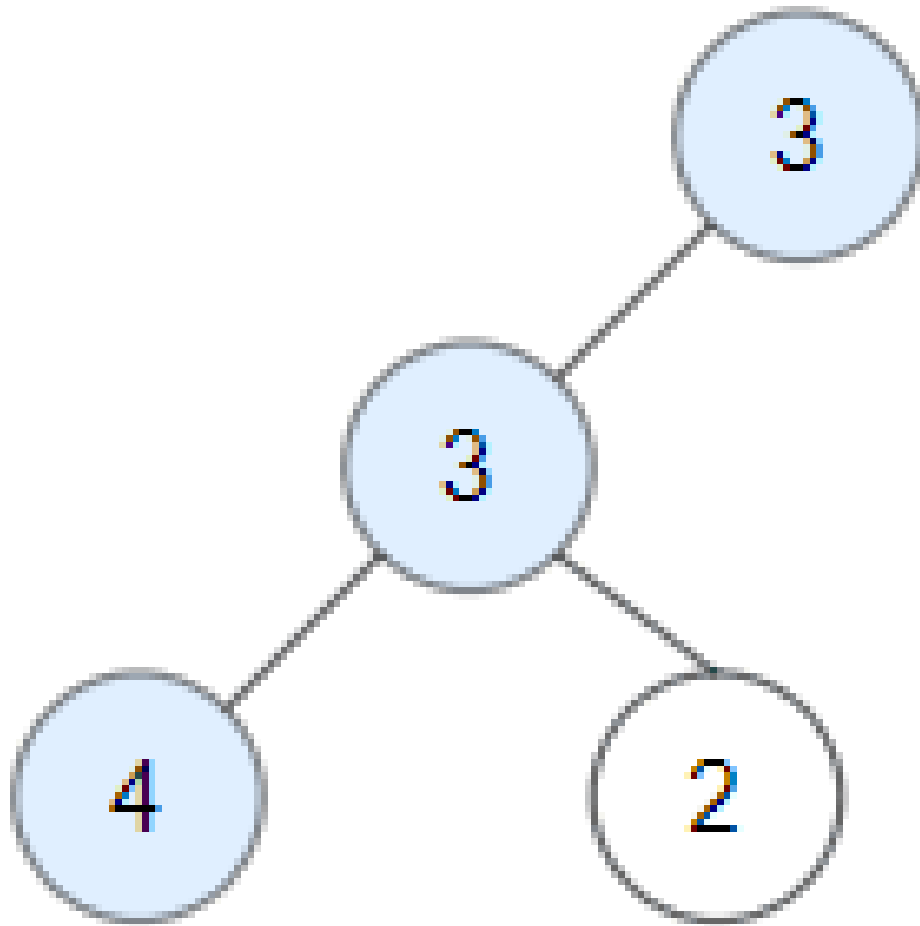
Input: root = [3,1,4,3,null,1,5]

Output: 4

Explanation: Nodes in blue are good.

Root Node (3) is always a good node.

Example 2:



Input: root = [3,3,null,4,2]

Output: 3

Explanation: Node 2 -> (3, 3, 2) is not good, because "3" is higher than it.

Example 3:

Input: root = [1]

Output: 1

Explanation: Root is considered as good.

Constraints:

- The number of nodes in the binary tree is in the range $[1, 10^5]$.
- Each node's value is between $[-10^4, 10^4]$.

Community Solutions (Python)

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def goodNodes(self, root: TreeNode) -> int:
        def dfs(root: TreeNode, mx: int):
            if root is None:
                return
            nonlocal ans
            if mx <= root.val:
                ans += 1
                mx = root.val
            dfs(root.left, mx)
            dfs(root.right, mx)

        ans = 0
        dfs(root, -1000000)

        return ans
```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def goodNodes(self, root: TreeNode) -> int:
        # Helper function to perform DFS traversal.

        def dfs(node, currentMax):
            # If we reach a null node, there are no further nodes to count.
            if not node:
                return 0

            # Check if current node is good.
            count = 1 if node.val >= currentMax else 0
            # Update the current maximum value.
            newMax = max(currentMax, node.val)

            # Recursively count good nodes in the left and right subtrees.
            count += dfs(node.left, newMax)
            count += dfs(node.right, newMax)
            return count

        # Start DFS traversal from root with initial maximum value as root's
        # value.
        return dfs(root, root.val)
```

◆ Quick Review

Key Insights

- Perform a tree traversal (DFS or BFS) from the root.
- Keep track of the maximum value encountered along the current path.

- A node is "good" if its value is greater than or equal to the maximum value seen so far.
- Update the maximum value as you traverse deeper in the tree.
- The root is always a good node.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes, since each node is visited exactly once.

Space Complexity: $O(h)$, where h is the height of the tree ($O(n)$ in the worst-case of a skewed tree).

Solution

We can solve this problem using Depth-First Search (DFS) recursion. Starting from the root, we pass along the current maximum value encountered on the path. For each node, we check if its value is greater than or equal to this maximum. If it is, we count it as a good node and update the maximum value if necessary. This process is continued recursively for both the left and right children of the node. Finally, we sum the counts from both subtrees.

Graphs

Round 2 · High · Medium · 1 question

Graphs

□ #684 Redundant Connection

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Depth-First Search · Breadth-First Search ·
Union-Find · Graph Theory
Lists: NeetCode 150

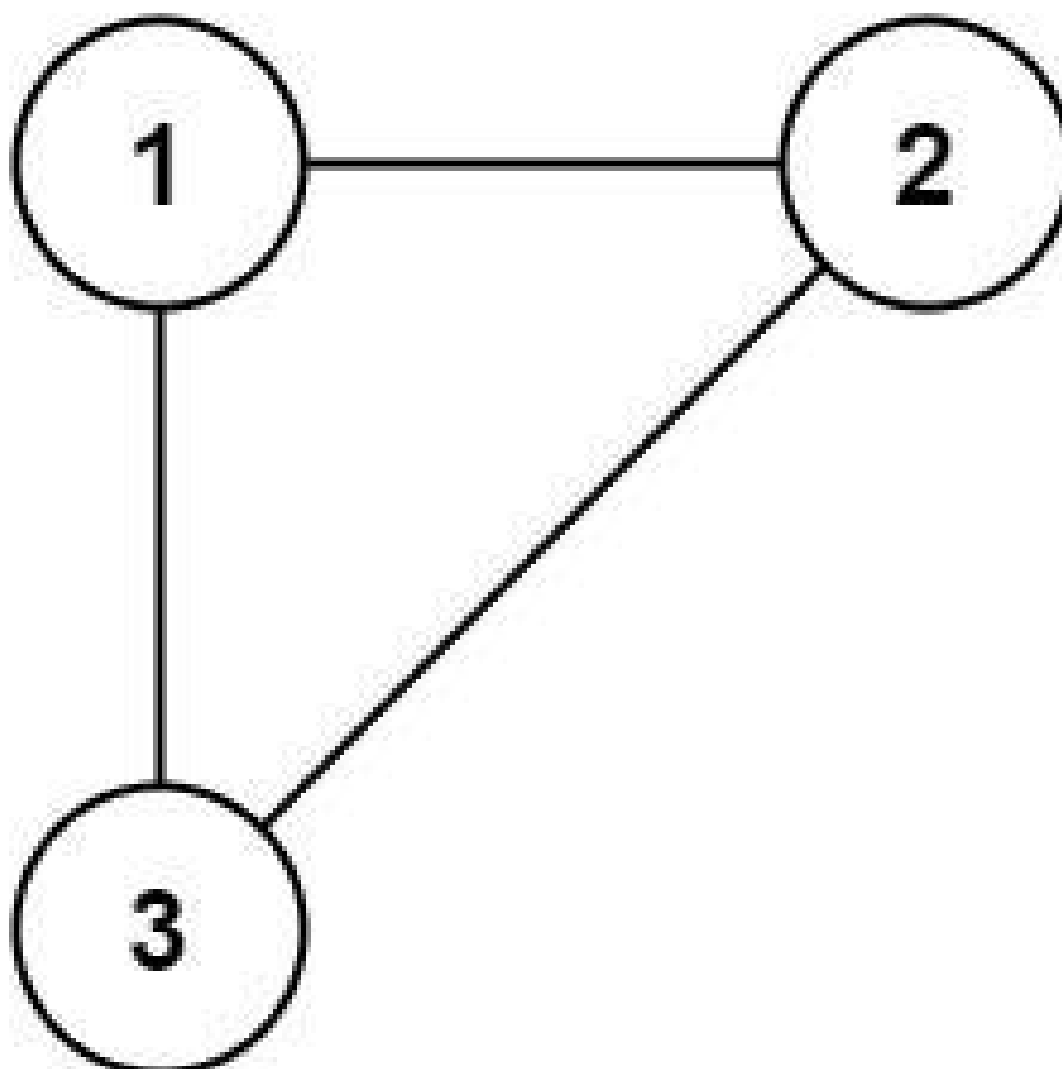
Problem

In this problem, a tree is an undirected graph that is connected and has no cycles.

You are given a graph that started as a tree with n nodes labeled from 1 to n , with one additional edge added. The added edge has two different vertices chosen from 1 to n , and was not an edge that already existed. The graph is represented as an array `edges` of length n where `edges[i] = [ai, bi]` indicates that there is an edge between nodes ai and bi in the graph.

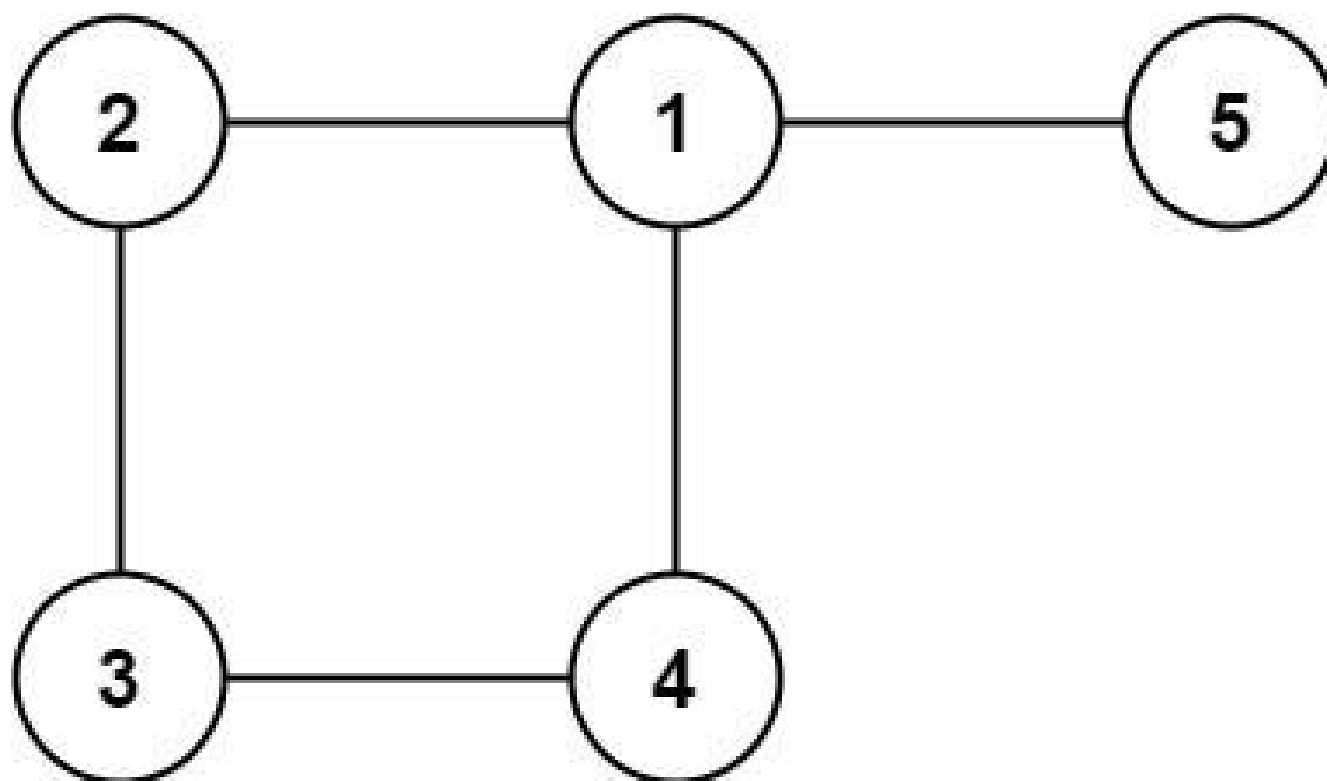
Return an edge that can be removed so that the resulting graph is a tree of n nodes. If there are multiple answers, return the answer that occurs last in the input.

Example 1:



Input: edges = [[1,2],[1,3],[2,3]]
Output: [2,3]

Example 2:



Input: edges = [[1,2],[2,3],[3,4],[1,4],[1,5]]

Output: [1,4]

Constraints:

- $n == \text{edges.length}$
- $3 \leq n \leq 1000$
- $\text{edges}[i].\text{length} == 2$
- $1 \leq a_i < b_i \leq \text{edges.length}$
- $a_i \neq b_i$
- There are no repeated edges.
- The given graph is connected.

Community Solutions (Python)

- WalkCC

```

class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> bool:
        i = self._find(u)
        j = self._find(v)
        if i == j:
            return False

        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1
        return True

    def _find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self._find(self.id[u])
        return self.id[u]

class Solution:
    def findRedundantConnection(self, edges: list[list[int]]) -> list[int]:
        uf = UnionFind(len(edges) + 1)

        for edge in edges:
            u, v = edge
            if not uf.unionByRank(u, v):
                return edge

```

• LeetDoocs

```
class Solution:
    def findRedundantConnection(self, edges: List[List[int]]) -> List[int]:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        p = list(range(len(edges)))
        for a, b in edges:
            pa, pb = find(a - 1), find(b - 1)

            if pa == pb:
                return [a, b]
            p[pa] = pb

class UnionFind:
    __slots__ = "p", "size"

    def __init__(self, n: int):
        self.p: List[int] = list(range(n))
        self.size: List[int] = [1] * n

    def find(self, x: int) -> int:
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])

        return self.p[x]

    def union(self, a: int, b: int) -> bool:
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
```

```

        self.p[pa] = pb
        self.size[pb] += self.size[pa]
    return True

```

```

class Solution:
    def findRedundantConnection(self, edges: List[List[int]]) -> List[int]:
        uf = UnionFind(len(edges))
        for a, b in edges:
            if not uf.union(a - 1, b - 1):
                return [a, b]

```

• SimplyLeet

Python solution using Union Find with path compression.

```

class UnionFind:
    def __init__(self, size):
        # Initialize parent pointers for each node.
        self.parent = list(range(size))

    def find(self, x):
        # Find the root of x with path compression.
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        # Union the sets of x and y.
        rootX = self.find(x)
        rootY = self.find(y)
        if rootX == rootY:
            return False # Cycle detected.
        self.parent[rootY] = rootX

```

```
        return True

def findRedundantConnection(edges):
    # Number of nodes is equal to len(edges) because edges were from a tree.
    n = len(edges)
    uf = UnionFind(n + 1) # Account for node labels starting at 1.
    redundant_edge = None
    for edge in edges:
        a, b = edge
        # If union returns False, a cycle is detected.

        if not uf.union(a, b):
            redundant_edge = edge
    return redundant_edge

# Example usage:
edges = [[1,2],[1,3],[2,3]]
print(findRedundantConnection(edges)) # Output: [2, 3]
```

◆ Quick Review

Key Insights

- The problem can be solved using a Union Find (Disjoint Set Union) data structure.
- As we iterate over each edge, we check whether the two vertices are already connected.
- If connecting the two vertices would create a cycle (i.e., they already belong to the same set), then the current edge is redundant.
- Alternatives like DFS or BFS are possible, but Union Find offers a more straightforward and

efficient solution.

Space & Time

Time Complexity: $O(n * \alpha(n))$ where $\alpha(n)$ is the inverse Ackermann function (almost constant in practice).

Space Complexity: $O(n)$, for storing the parent array used in the Union Find structure.

Solution

We use the Union Find algorithm by maintaining an array where each node points to its parent. For each edge, we perform a find operation on both endpoints:

– If the endpoints belong to the same set, adding this edge would create a cycle and it is marked as redundant. – Otherwise, we union the two sets. This algorithm efficiently detects the extra edge that introduces a cycle in the graph.

Advanced Graphs

Round 2 · High · Medium · 2 questions

Advanced Graphs

□ #743 Network Delay Time

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Depth-First Search · Breadth-First Search ·
Graph Theory · Heap (Priority Queue) · Shortest
Path

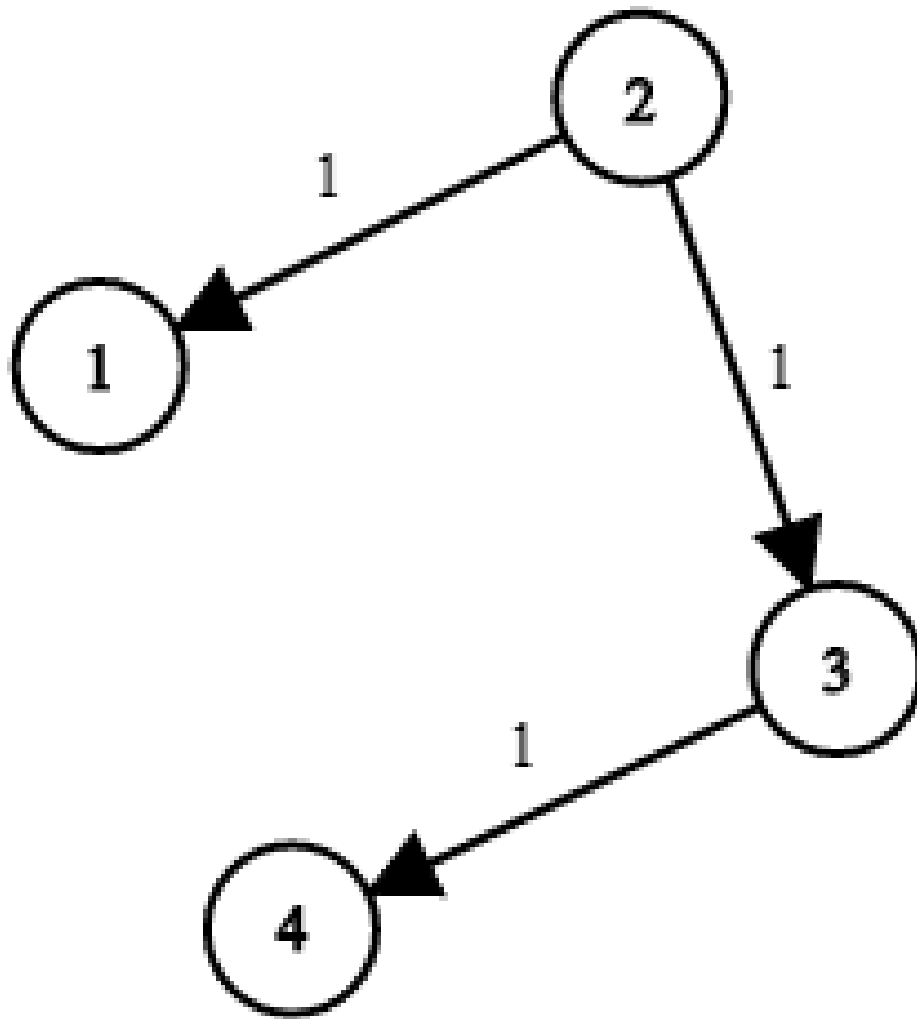
Lists: NeetCode 150

Problem

You are given a network of n nodes, labeled from 1 to n . You are also given times, a list of travel times as directed edges $\text{times}[i] = (u_i, v_i, w_i)$, where u_i is the source node, v_i is the target node, and w_i is the time it takes for a signal to travel from source to target.

We will send a signal from a given node k . Return the minimum time it takes for all the n nodes to receive the signal. If it is impossible for all the n nodes to receive the signal, return -1 .

Example 1:



Input: times = [[2,1,1],[2,3,1],[3,4,1]], n = 4, k = 2
Output: 2

Example 2:

Input: times = [[1,2,1]], n = 2, k = 1
Output: 1

Example 3:

Input: times = [[1,2,1]], n = 2, k = 2
Output: -1

Constraints:

- $1 \leq k \leq n \leq 100$

- $1 \leq \text{times.length} \leq 6000$
- $\text{times}[i].\text{length} == 3$
- $1 \leq u_i, v_i \leq n$
- $u_i \neq v_i$
- $0 \leq w_i \leq 100$
- All the pairs (u_i, v_i) are unique. (i.e., no multiple edges.)

Community Solutions (Python)

• WalkCC

```
class Solution:
    def networkDelayTime(self, times: list[list[int]], n: int, k: int) -> int:
        graph = [[] for _ in range(n)]

        for u, v, w in times:
            graph[u - 1].append((v - 1, w))

        return self._dijkstra(graph, k - 1)

    def _dijkstra(self, graph: list[list[tuple[int, int]]], src: int) -> int:

        dist = [math.inf] * len(graph)

        dist[src] = 0
        minHeap = [(dist[src], src)] # (d, u)

        while minHeap:
            d, u = heapq.heappop(minHeap)
            if d > dist[u]:
                continue
            for v, w in graph[u]:
```

```

        if d + w < dist[v]:
            dist[v] = d + w
            heapq.heappush(minHeap, (dist[v], v))

    maxDist = max(dist)
    return maxDist if maxDist != math.inf else -1

```

• LeetCode

```

class Solution:
    def networkDelayTime(self, times: List[List[int]], n: int, k: int) -> int:
        g = [[inf] * n for _ in range(n)]
        for u, v, w in times:
            g[u - 1][v - 1] = w
        dist = [inf] * n
        dist[k - 1] = 0
        vis = [False] * n
        for _ in range(n):
            t = -1
            for j in range(n):
                if not vis[j] and (t == -1 or dist[t] > dist[j]):
                    t = j
            vis[t] = True
            for j in range(n):
                dist[j] = min(dist[j], dist[t] + g[t][j])
        ans = max(dist)
        return -1 if ans == inf else ans

```

• SimplyLeet

```

import heapq

def networkDelayTime(times, n, k):
    # Build the graph as an adjacency list
    graph = {i: [] for i in range(1, n + 1)}
    for u, v, w in times:
        graph[u].append((v, w))

    # Initialize distances: infinity for all nodes except the starting node k
    # with distance 0
    dist = {i: float('inf') for i in range(1, n + 1)}

```

```
dist[k] = 0
# Use a heap as a priority queue storing (distance, node) pairs
heap = [(0, k)]

while heap:
    current_dist, node = heapq.heappop(heap)
    # If we have already found a better path, skip
    if current_dist > dist[node]:
        continue
    # Update the path for neighbors

    for neighbor, weight in graph[node]:
        new_dist = current_dist + weight
        if new_dist < dist[neighbor]:
            dist[neighbor] = new_dist
            heapq.heappush(heap, (new_dist, neighbor))

    # Maximum distance among all nodes represents the time taken for the signal
    # to reach every node
    max_time = max(dist.values())
    return max_time if max_time != float('inf') else -1

# Example usage:
print(networkDelayTime([[2,1,1],[2,3,1],[3,4,1]], 4, 2))
```

◆ Quick Review

Key Insights

- This is a single-source shortest path problem on a directed graph.
- Since all edge weights are non-negative, Dijkstra's algorithm is well-suited for this problem.
- Construct an adjacency list for the graph.

- Utilize a min-heap (priority queue) to efficiently select the next node with the smallest cumulative travel time.
- The answer is the maximum travel time among all reachable nodes; if any node is unreachable, return -1 .

Space & Time

Time Complexity: $O(E \log V)$ where E is the number of edges and V is the number of nodes.

Space Complexity: $O(V + E)$ due to the adjacency list and distance storage.

Solution

We solve the problem using Dijkstra's algorithm. First, an adjacency list is built using the input times array. A distance map (or array) records the shortest known distance from the starting node k to all other nodes, initialized to infinity except for k which is set to 0. A min-heap stores nodes prioritized by their current shortest distance.

The algorithm repeatedly extracts the node with the smallest distance from the heap. For each neighbor of the current node, it checks if

a shorter path is found via the current node; if so, the neighbor's distance is updated and it is added to the heap. Once all nodes are processed, the answer is the maximum distance from the k node. If any node remains unreachable (has distance equal to infinity), return -1.

Advanced Graphs

□ #1584 Min Cost to Connect All Points

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Union-Find · Graph Theory · Minimum
Spanning Tree

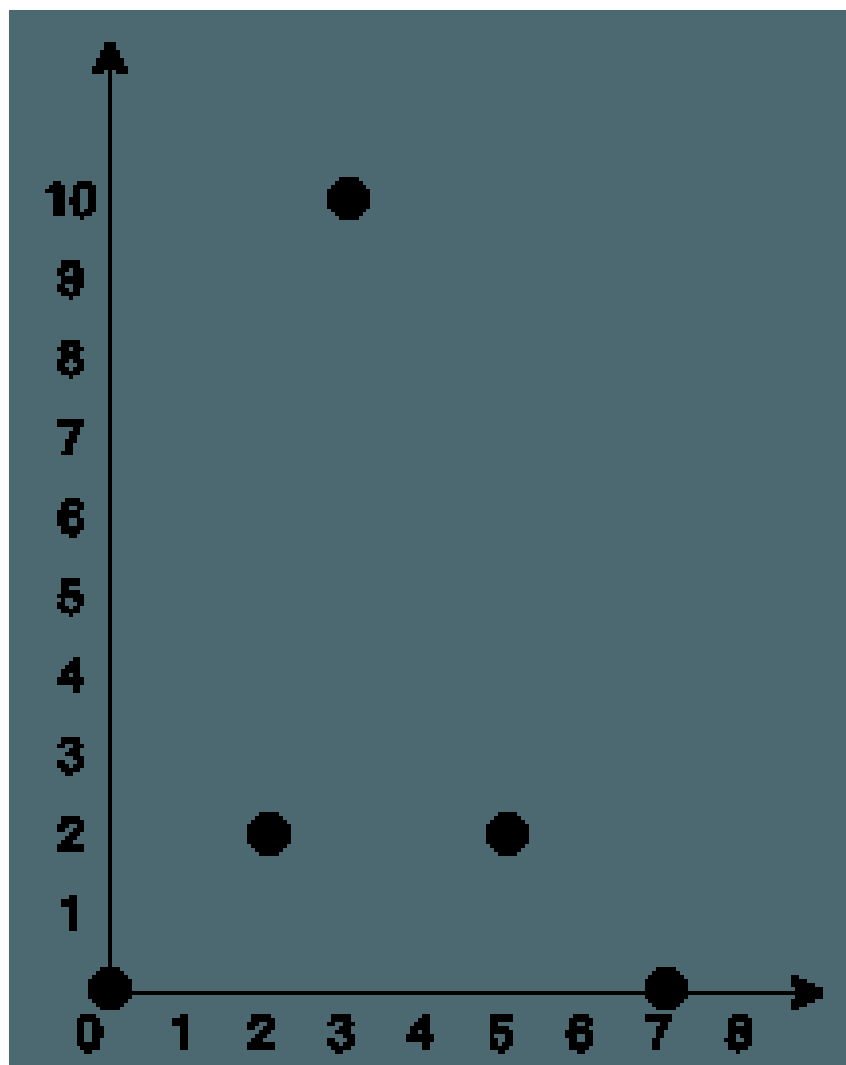
Lists: NeetCode 150

Problem

You are given an array `points` representing integer coordinates of some points on a 2D-plane, where `points[i] = [xi, yi]`.

The cost of connecting two points `[xi, yi]` and `[xj, yj]` is the manhattan distance between them: $|xi - xj| + |yi - yj|$, where $|val|$ denotes the absolute value of `val`.

Return the minimum cost to make all points connected. All points are connected if there is exactly one simple path between any two points.
Example 1:



Input: points = [[0,0],[2,2],[3,10],[5,2],[7,0]]

Output: 20

Explanation:

We can connect the points as shown above to get the minimum cost of 20.
Notice that there is a unique path between every pair of points.

Example 2:

Input: points = [[3,12],[-2,5],[-4,1]]

Output: 18

Constraints:

- $1 \leq \text{points.length} \leq 1000$
- $-106 \leq x_i, y_i \leq 106$

- All pairs (xi, yi) are distinct.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minCostConnectPoints(self, points: List[int]) -> int:
        # dist[i] := the minimum distance to connect the points[i]
        dist = [math.inf] * len(points)
        ans = 0

        for i in range(len(points) - 1):
            for j in range(i + 1, len(points)):
                # Try to connect the points[i] with the points[j].
                dist[j] = min(dist[j], abs(points[i][0] - points[j][0]) +
                               abs(points[i][1] - points[j][1]))
                # Swap the points[j] (the point with the minimum distance) with the
                # points[i + 1].
                if dist[j] < dist[i + 1]:
                    points[j], points[i + 1] = points[i + 1], points[j]
                    dist[j], dist[i + 1] = dist[i + 1], dist[j]
            ans += dist[i + 1]

        return ans
```

• LeetDoocs

```
class Solution:
    def minCostConnectPoints(self, points: List[List[int]]) -> int:
        n = len(points)
        g = [[0] * n for _ in range(n)]
        dist = [inf] * n
        vis = [False] * n
        for i, (x1, y1) in enumerate(points):
            for j in range(i + 1, n):
                x2, y2 = points[j]
                t = abs(x1 - x2) + abs(y1 - y2)
```

```

        g[i][j] = g[j][i] = t
    dist[0] = 0
    ans = 0
    for _ in range(n):
        i = -1
        for j in range(n):
            if not vis[j] and (i == -1 or dist[j] < dist[i]):
                i = j
        vis[i] = True
        ans += dist[i]

```

```

        for j in range(n):
            if not vis[j]:
                dist[j] = min(dist[j], g[i][j])
    return ans

```

```

class Solution:
    def minCostConnectPoints(self, points: List[List[int]]) -> int:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        n = len(points)
        g = []
        for i, (x1, y1) in enumerate(points):

```

```

            for j in range(i + 1, n):
                x2, y2 = points[j]
                t = abs(x1 - x2) + abs(y1 - y2)
                g.append((t, i, j))
        p = list(range(n))
        ans = 0
        for cost, i, j in sorted(g):
            pa, pb = find(i), find(j)
            if pa == pb:
                continue

```

```

        p[pa] = pb
        ans += cost
        n -= 1
        if n == 1:
            break
    return ans

```

• SimplyLeet

Python solution using Prim's algorithm

```
class Solution:
```

```
    def minCostConnectPoints(self, points):
```

```
        # number of points
```

```
        n = len(points)
```

```
        # initialize cost for each point as infinity, except the first point
```

```
        minCost = [float('inf')] * n
```

```
        minCost[0] = 0 # start from the first point
```

```
        # keep track of which points are included in the MST
```

```
        inMST = [False] * n
```

```
        # total cost of connecting all points
```

```
        total_cost = 0
```

```
        for _ in range(n):
```

```
            # select the next point with the smallest connection cost
```

```
            curr = -1
```

```
            curr_cost = float('inf')
```

```
            for i in range(n):
```

```
                if not inMST[i] and minCost[i] < curr_cost:
```

```
                    curr = i
```

```
                    curr_cost = minCost[i]
```

```
            # add the cost of the chosen point
```

```
            total_cost += curr_cost
```

```
            inMST[curr] = True
```

```
        # update the connection costs for the remaining points
```

```
for next_point in range(n):
    if not inMST[next_point]:
        # compute the Manhattan distance between the current and
next point
        (x1, y1), (x2, y2) = points[curr], points[next_point]
        dist = abs(x1 - x2) + abs(y1 - y2)
        if dist < minCost[next_point]:
            minCost[next_point] = dist

return total_cost
```

```
# Example usage:
# sol = Solution()
# print(sol.minCostConnectPoints([[0,0],[2,2],[3,10],[5,2],[7,0]]))
```

◆ Quick Review

Key Insights

- The problem can be reinterpreted as finding a Minimum Spanning Tree (MST) on a complete graph.
- Manhattan distance is used to calculate the weight/cost between every pair of vertices.
- Prim's or Kruskal's algorithm can be applied to solve the MST problem.
- A direct construction of the complete graph is unnecessary; distances can be computed on the fly, which is efficient given the problem constraints.

Space & Time

Time Complexity: $O(n^2)$

Space Complexity: $O(n)$

Solution

We can solve the problem using Prim's algorithm. We start from an arbitrary point (for example, the first point) and iteratively add the point that has the smallest distance to the growing MST. Instead of explicitly constructing all the edges, we compute the Manhattan distance on the fly between the current MST and the remaining points. An array is used to track the minimum connection cost for each point that is not yet included in the tree. At each iteration, we select the point with the smallest cost to add into the MST and update the costs for the remaining points. This approach guarantees that we connect all points with the minimum total cost.

Round 3 · High · Hard

Round 3

High Priority · Hard

3 questions · 2 patterns

Intervals #1851

Advanced Graphs #332  #778

Intervals

Round 3 · High · Hard · 1 question

Intervals

#1851 Minimum Interval to Include Each Query **HAR**

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Sweep Line · Sorting ·
Heap (Priority Queue)

Lists: NeetCode 150

Problem

You are given a 2D integer array `intervals`, where `intervals[i] = [lefti, righti]` describes the *i*th interval starting at `lefti` and ending at `righti` (inclusive). The size of an interval is defined as the number of integers it contains, or more formally `righti - lefti + 1`.

You are also given an integer array `queries`. The answer to the *j*th query is the size of the smallest interval *i* such that `lefti ≤ queries[j] ≤ righti`. If no such interval exists, the answer is `-1`.

Return an array containing the answers to the queries.

Example 1:

Input: intervals = [[1,4],[2,4],[3,6],[4,4]], queries = [2,3,4,5]

Output: [3,3,1,4]

Explanation: The queries are processed as follows:

- Query = 2: The interval [2,4] is the smallest interval containing 2. The answer is $4 - 2 + 1 = 3$.
- Query = 3: The interval [2,4] is the smallest interval containing 3. The answer is $4 - 2 + 1 = 3$.
- Query = 4: The interval [4,4] is the smallest interval containing 4. The answer is $4 - 4 + 1 = 1$.
- Query = 5: The interval [3,6] is the smallest interval containing 5. The answer is $6 - 3 + 1 = 4$.

Example 2:

Input: intervals = [[2,3],[2,5],[1,8],[20,25]], queries = [2,19,5,22]

Output: [2,-1,4,6]

Explanation: The queries are processed as follows:

- Query = 2: The interval [2,3] is the smallest interval containing 2. The answer is $3 - 2 + 1 = 2$.
- Query = 19: None of the intervals contain 19. The answer is -1.
- Query = 5: The interval [2,5] is the smallest interval containing 5. The answer is $5 - 2 + 1 = 4$.
- Query = 22: The interval [20,25] is the smallest interval containing 22. The answer is $25 - 20 + 1 = 6$.

Constraints:

- $1 \leq \text{intervals.length} \leq 105$
- $1 \leq \text{queries.length} \leq 105$
- $\text{intervals}[i].\text{length} == 2$
- $1 \leq \text{lefti} \leq \text{righti} \leq 10^7$
- $1 \leq \text{queries}[j] \leq 10^7$

Community Solutions (Python)

- LeetDoocs

```

class Solution:
    def minInterval(self, intervals: List[List[int]], queries: List[int]) -> List[int]:
        n, m = len(intervals), len(queries)
        intervals.sort()
        queries = sorted((x, i) for i, x in enumerate(queries))
        ans = [-1] * m
        pq = []
        i = 0
        for x, j in queries:
            while i < n and intervals[i][0] <= x:
                a, b = intervals[i]
                heappush(pq, (b - a + 1, b))
                i += 1
            while pq and pq[0][1] < x:
                heappop(pq)
            if pq:
                ans[j] = pq[0][0]
        return ans

```

• SimplyLeet

```

import heapq

def minInterval(intervals, queries):
    # Sort intervals by their starting point.
    intervals.sort(key=lambda itv: itv[0])

    # Prepare queries sorted by their value, keeping track of original index.
    sorted_queries = sorted([(q, i) for i, q in enumerate(queries)])

    result = [-1] * len(queries)

```

```

min_heap = []
i = 0 # for intervals

# Process each query in ascending order.
for q, qi in sorted_queries:
    # Add all intervals that start <= current query.
    while i < len(intervals) and intervals[i][0] <= q:
        start, end = intervals[i]
        # Compute the size of the interval.
        size = end - start + 1

        # Push tuple: (size, end) into the min-heap.
        heapq.heappush(min_heap, (size, end))
        i += 1
    # Remove intervals that do not cover the current query.
    while min_heap and min_heap[0][1] < q:
        heapq.heappop(min_heap)
    # The top of the heap (if any) is the smallest interval covering the
    query.
    if min_heap:
        result[qi] = min_heap[0][0]
    return result

# Example usage
intervals = [[1,4],[2,4],[3,6],[4,4]]
queries = [2,3,4,5]
print(minInterval(intervals, queries)) # Output: [3, 3, 1, 4]

```

◆ Quick Review

Key Insights

- Sort the intervals by their start values to efficiently determine when an interval becomes relevant for a given query.
- Process queries in increasing order so that intervals can be added to a data structure

(min-heap) as they "start" to cover queries.

- Use a min-heap keyed by the size (length) of the interval to quickly retrieve the smallest interval currently containing the query.
- Remove any intervals from the heap that no longer cover the current query (i.e., their end is less than the query).
- Maintain the original order of queries by storing the indices while sorting.

Space & Time

Time Complexity: $O((N + Q) \log N)$ where N is the number of intervals and Q is the number of queries. Sorting and heap operations dominate the complexity.

Space Complexity: $O(N + Q)$ for storing intervals, queries, and the heap.

Solution

The solution uses a two-step approach:

- Sort the queries while keeping track of their original indices.
- Sort intervals by their starting values. For each query (in increasing order), add intervals that start before or at this query to a min-heap, where each entry is the size of the interval along with its end value.

Then, remove intervals from the heap that do not cover the query (their end value is less than the query). The heap's top element (if available) holds the smallest interval size that contains the query. – Place the answer in a result array at the position corresponding to the original query order.

Data structures used: Sorting arrays and a min-heap for efficient retrieval and removal.

Advanced Graphs

Round 3 · High · Hard · 2 questions

Advanced Graphs

#332 Reconstruct Itinerary

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Depth-First Search · Graph
Theory · Sorting · Heap (Priority Queue) ·
Eulerian Circuit

Lists: NeetCode 150

Problem

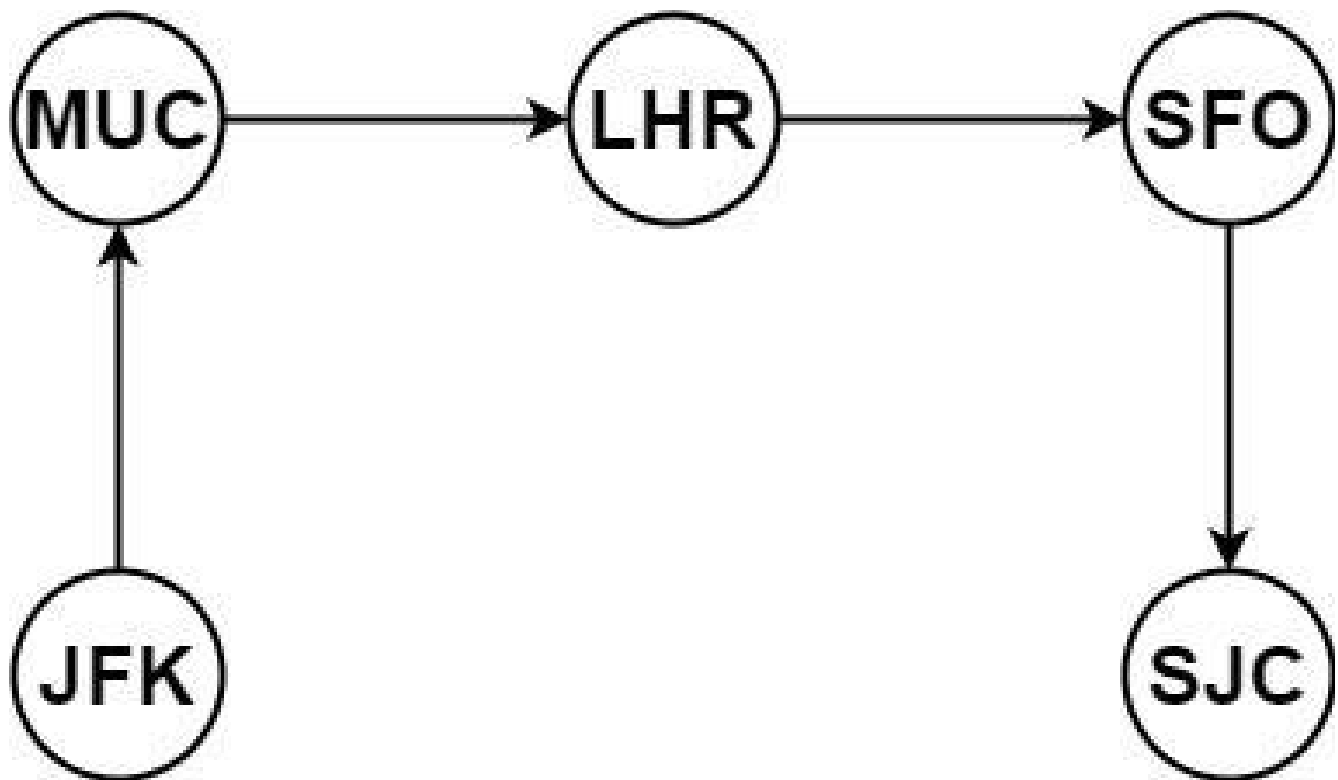
You are given a list of airline tickets where $\text{tickets}[i] = [\text{from}_i, \text{to}_i]$ represent the departure and the arrival airports of one flight.

Reconstruct the itinerary in order and return it. All of the tickets belong to a man who departs from "JFK", thus, the itinerary must begin with "JFK". If there are multiple valid itineraries, you should return the itinerary that has the smallest lexical order when read as a single string.

- For example, the itinerary ["JFK", "LGA"] has a smaller lexical order than ["JFK", "LGB"].

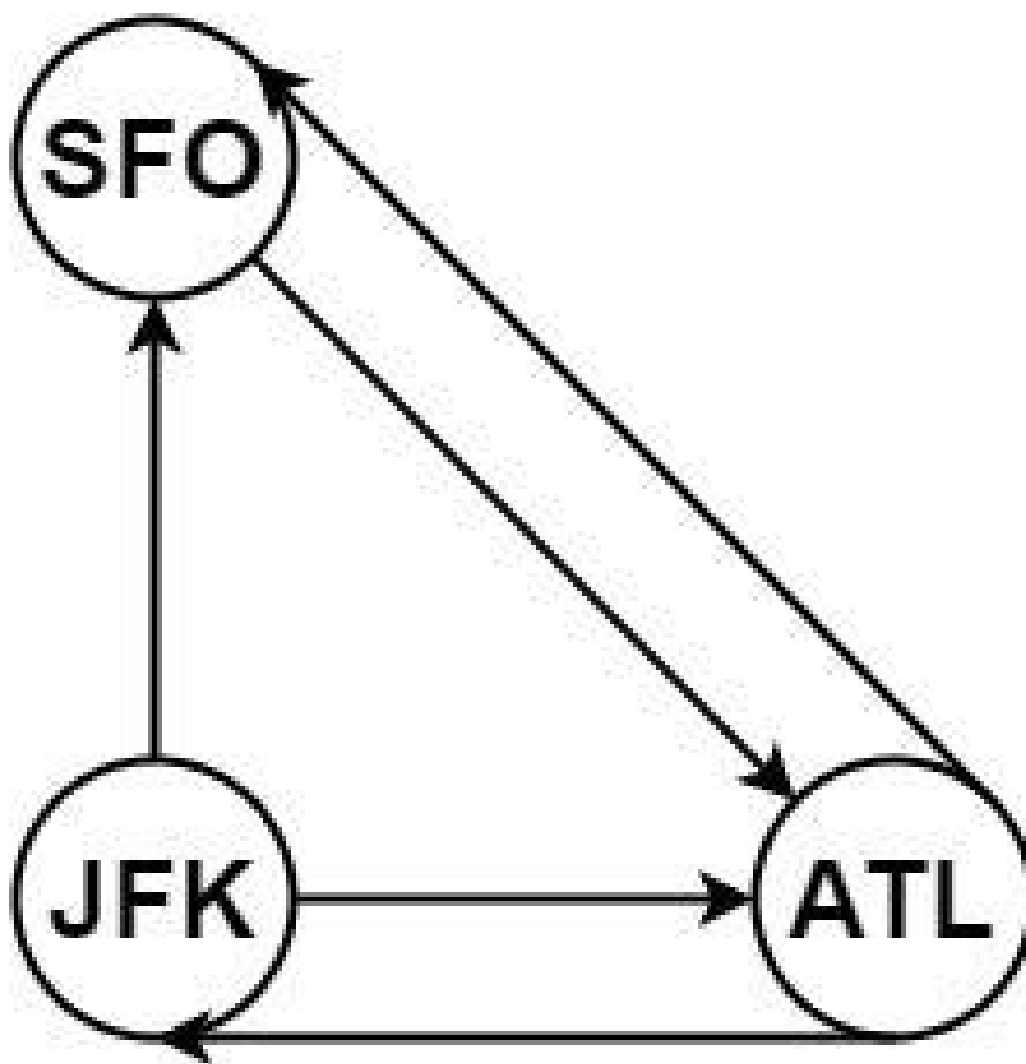
You may assume all tickets form at least one valid itinerary. You must use all the tickets once and only once.

Example 1:



Input: tickets = [["MUC", "LHR"], ["JFK", "MUC"], ["SFO", "SJC"], ["LHR", "SFO"]]
Output: ["JFK", "MUC", "LHR", "SFO", "SJC"]

Example 2:



Input: tickets =
[["JFK", "SFO"], ["JFK", "ATL"], ["SFO", "ATL"], ["ATL", "JFK"], ["ATL", "SFO"]]
Output: ["JFK", "ATL", "JFK", "SFO", "ATL", "SFO"]
Explanation: Another possible reconstruction is
["JFK", "SFO", "ATL", "JFK", "ATL", "SFO"] but it is larger in lexical order.

Constraints:

- $1 \leq \text{tickets.length} \leq 300$
- $\text{tickets}[i].\text{length} == 2$
- $\text{fromi.length} == 3$
- $\text{toi.length} == 3$

- fromi and toi consist of uppercase English letters.
- fromi != toi

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findItinerary(self, tickets: list[list[str]]) -> list[str]:
        ans = []
        graph = collections.defaultdict(list)

        for a, b in reversed(sorted(tickets)):
            graph[a].append(b)

        def dfs(u: str) -> None:
            while u in graph and graph[u]:

                dfs(graph[u].pop())
            ans.append(u)

        dfs('JFK')
        return ans[::-1]
```

• LeetDoocs

```
class Solution:
    def findItinerary(self, tickets: List[List[str]]) -> List[str]:
        def dfs(f: str):
            while g[f]:
                dfs(g[f].pop())
            ans.append(f)

        g = defaultdict(list)
        for f, t in sorted(tickets, reverse=True):
            g[f].append(t)
```

```
ans = []
dfs("JFK")
return ans[::-1]
```

• SimplyLeet

```
def findItinerary(tickets):
    # Build the graph as an adjacency list.
    graph = {}
    for src, dst in tickets:
        if src not in graph:
            graph[src] = []
        graph[src].append(dst)
    # Sort the destination lists in reverse lexicographical order.
    for src in graph:
        graph[src].sort(reverse=True)
```

```
itinerary = []
```

```
def dfs(airport):
    # Explore all flights out of the current airport.
    while airport in graph and graph[airport]:
        next_dest = graph[airport].pop()
        dfs(next_dest)
    itinerary.append(airport)
```

```
# Start DFS from 'JFK'
dfs("JFK")
# The itinerary is constructed in reverse.
return itinerary[::-1]
```

```
# Example usage:
tickets = [["JFK","SFO"], ["JFK","ATL"], ["SFO","ATL"], ["ATL","JFK"], ["ATL",
"SFO"]]
print(findItinerary(tickets))
```

◆ Quick Review

Key Insights

- Model the tickets as a directed graph where airports are nodes.
- The problem is akin to finding an Eulerian path through the graph.
- Use Depth-First Search (DFS) with Hierholzer's algorithm to construct the itinerary.
- To ensure lexical order, sort the destination lists in reverse order so that the smallest lexical destination can be popped during DFS.
- Build the itinerary in reverse during backtracking and then reverse it to get the final order.

Space & Time

Time Complexity: $O(E \log E)$, where E is the number of tickets, due to the sorting of edges.

Space Complexity: $O(E)$, to store the graph and recursion stack.

Solution

Construct an adjacency list (graph) mapping each departure airport to a list of destination airports sorted in reverse lexicographical order. Perform DFS starting from "JFK". During DFS, traverse while there are destinations

available from the current airport. Append airports to the itinerary as you backtrack. Reverse the built itinerary at the end to obtain the correct order.

Advanced Graphs

#778 Swim in Rising Water

HAR

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Depth-First Search ·
Breadth-First Search · Union-Find · Heap
(Priority Queue) · Matrix

Lists: NeetCode 150

Problem

You are given an $n \times n$ integer matrix grid where each value $\text{grid}[i][j]$ represents the elevation at that point (i, j) .

It starts raining, and water gradually rises over time. At time t , the water level is t , meaning any cell with elevation less than equal to t is submerged or reachable.

You can swim from a square to another 4-directionally adjacent square if and only if the elevation of both squares individually are at most t . You can swim infinite distances in zero time. Of course, you must stay within the boundaries of the grid during your swim.

Return the minimum time until you can reach the bottom right square ($n - 1, n - 1$) if you start at the top left square $(0, 0)$.

Example 1:

0	2
1	3

Input: `grid = [[0,2],[1,3]]`

Output: 3

Explanation:

At time 0, you are in grid location $(0, 0)$.

You cannot go anywhere else because 4-directionally adjacent neighbors have a higher elevation than $t = 0$.

You cannot reach point $(1, 1)$ until time 3.

When the depth of water is 3, we can swim anywhere inside the grid.

Example 2:

0	1	2	3	4
24	23	22	21	5
12	13	14	15	16
11	17	18	19	20
10	9	8	7	6

Input: grid =

[[0,1,2,3,4],[24,23,22,21,5],[12,13,14,15,16],[11,17,18,19,20],[10,9,8,7,6]]

Output: 16

Explanation: The final route is shown.

We need to wait until time 16 so that (0, 0) and (4, 4) are connected.

Constraints:

- $n == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq n \leq 50$
- $0 \leq \text{grid}[i][j] < n^2$

- Each value `grid[i][j]` is unique.

Community Solutions (Python)

• LeetCode

```
class Solution:
    def swimInWater(self, grid: List[List[int]]) -> int:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        n = len(grid)
        m = n * n
        p = list(range(m))

        hi = [0] * m
        for i, row in enumerate(grid):
            for j, h in enumerate(row):
                hi[h] = i * n + j
        dirs = (-1, 0, 1, 0, -1)
        for t in range(m):
            x, y = divmod(hi[t], n)
            for dx, dy in pairwise(dirs):
                nx, ny = x + dx, y + dy
                if 0 <= nx < n and 0 <= ny < n and grid[nx][ny] <= t:
                    p[find(x * n + y)] = find(nx * n + ny)
            if find(0) == find(m - 1):
                return t
        return 0
```

• SimplyLeet

```

import heapq

def swimInWater(grid):
    n = len(grid)
    # Initialize a visited set to track cells we have processed.
    visited = [[False] * n for _ in range(n)]
    # Min-heap: (current max elevation along the path, row, col)
    min_heap = [(grid[0][0], 0, 0)]
    visited[0][0] = True
    # Directions for moving up, down, left, and right.

    directions = [(1,0), (-1,0), (0,1), (0,-1)]

    while min_heap:
        time, r, c = heapq.heappop(min_heap)
        # If we reach the bottom-right, return the maximum elevation
        encountered along the path.
        if r == n - 1 and c == n - 1:
            return time
        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            # Check boundaries and if the cell is not already visited.

            if 0 <= nr < n and 0 <= nc < n and not visited[nr][nc]:
                visited[nr][nc] = True
                # The required time is the maximum of the current time and the
                elevation at the new cell.
                new_time = max(time, grid[nr][nc])
                heapq.heappush(min_heap, (new_time, nr, nc))

    return -1 # Should not reach here if input is as per constraints.

# Example Usage:
grid = [[0,2],[1,3]]

print(swimInWater(grid)) # Output: 3

```

◆ Quick Review

Key Insights

- The problem can be viewed as finding a path from the top-left to bottom-right where the cost is the maximum elevation encountered on that path.
- A greedy approach using Dijkstra's algorithm (or a min-heap based BFS) ensures that we always consider the next move with the smallest current elevation requirement.
- Once the destination is reached, the maximum elevation along that path is the minimum time required.
- Alternative approaches include binary search on time combined with a DFS/BFS check to see if a valid path exists.

Space & Time

Time Complexity: $O(n^2 \log(n^2))$ which simplifies to $O(n^2 \log n)$ since each cell is processed once and we use a priority queue.

Space Complexity: $O(n^2)$ due to the visited matrix and the priority queue.

Solution

We can solve the problem using a min-heap based BFS (Dijkstra's algorithm). Start at cell (0, 0) with its elevation as the initial cost, and

use a min-heap to always expand the cell with the lowest required water level. For each popped cell, update the maximum elevation encountered on the current path. When reaching the target cell ($n - 1, n - 1$), this value represents the least time needed. We maintain a visited matrix to avoid reprocessing cells.

Round 4 · Mid · Easy

Round 4

Mid Priority · Easy

2 questions · 1 pattern

Heap / Priority Queue #703  #1046

Heap / Priority Queue

Round 4 · Mid · Easy · 2 questions

Heap / Priority Queue

#703 Kth Largest Element in a Stream

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · Design · Binary Search Tree · Heap
(Priority Queue) · Binary Tree · Data Stream
Lists: NeetCode 150

Problem

You are part of a university admissions office and need to keep track of the k th highest test score from applicants in real-time. This helps to determine cut-off marks for interviews and admissions dynamically as new applicants submit their scores.

You are tasked to implement a class which, for a given integer k , maintains a stream of test scores and continuously returns the k th highest test score after a new score has been submitted. More specifically, we are looking for the k th highest score in the sorted list of all scores.

Implement the `KthLargest` class:

- `KthLargest(int k, int[] nums)` Initializes the object with the integer k and the stream of

test scores nums.

- **int add(int val)** Adds a new test score val to the stream and returns the element representing the kth largest element in the pool of test scores so far.

Example 1:

Input: ["KthLargest", "add", "add", "add", "add", "add"]
[[3, [4, 5, 8, 2]], [3], [5], [10], [9], [4]]

Output: [null, 4, 5, 5, 8, 8]

Explanation:

```
KthLargest kthLargest = new KthLargest(3, [4, 5, 8, 2]);  
kthLargest.add(3); // return 4  
kthLargest.add(5); // return 5  
kthLargest.add(10); // return 5  
kthLargest.add(9); // return 8  
kthLargest.add(4); // return 8
```

Example 2:

Input: ["KthLargest", "add", "add", "add", "add"]
[[4, [7, 7, 7, 7, 8, 3]], [2], [10], [9], [9]]

Output: [null, 7, 7, 7, 8]

Explanation:

Constraints:

- $0 \leq \text{nums.length} \leq 10^4$
- $1 \leq k \leq \text{nums.length} + 1$

- $-104 \leq \text{nums}[i] \leq 104$
- $-104 \leq \text{val} \leq 104$
- At most 104 calls will be made to add.

Community Solutions (Python)

• LeetCode

```
class KthLargest:

    def __init__(self, k: int, nums: List[int]):
        self.k = k
        self.min_q = []
        for x in nums:
            self.add(x)

    def add(self, val: int) -> int:
        heappush(self.min_q, val)

        if len(self.min_q) > self.k:
            heappop(self.min_q)
        return self.min_q[0]

# Your KthLargest object will be instantiated and called as such:
# obj = KthLargest(k, nums)
# param_1 = obj.add(val)
```

• SimplyLeet

```
import heapq

class KthLargest:
    def __init__(self, k, nums):
        # Save kth value and initialize the min-heap.
        self.k = k
        self.min_heap = nums
        heapq.heapify(self.min_heap)
        # Reduce the heap size to exactly k elements.
        while len(self.min_heap) > k:
```

```
        heapq.heappop(self.min_heap)

    def add(self, val):
        if len(self.min_heap) < self.k:
            heapq.heappush(self.min_heap, val)
        elif val > self.min_heap[0]:
            heapq.heapreplace(self.min_heap, val)
        # The kth largest element is at the heap root.
        return self.min_heap[0]
```

```
# Example usage:
# kthLargest = KthLargest(3, [4, 5, 8, 2])
# print(kthLargest.add(3)) # returns 4
# print(kthLargest.add(5)) # returns 5
# print(kthLargest.add(10)) # returns 5
# print(kthLargest.add(9)) # returns 8
# print(kthLargest.add(4)) # returns 8
```

◆ Quick Review

Key Insights

- Use a min-heap (priority queue) of fixed size k .
- The heap always contains the k largest elements; the smallest element in the heap is the k th largest overall.
- When a new test score is added:
- If the heap has less than k elements, simply add the score.
- If the heap is full and the new score is greater than the smallest (heap root), replace

the root.

- Each add operation runs in $O(\log k)$ time.

Space & Time

Time Complexity: $O(\log k)$ per add operation, $O(n \log k)$ to build the initial heap.

Space Complexity: $O(k)$ for maintaining the heap.

Solution

We use a min-heap to track the k largest numbers. When initializing, we add numbers from the given list to the heap, ensuring not to exceed a size of k . For each add operation, if the new number is greater than the heap minimum, we replace the minimum. The k th largest number is always accessible as the minimum element in the heap.

Heap / Priority Queue

#1046 Last Stone Weight

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Heap (Priority Queue)

Lists: NeetCode 150

Problem

You are given an array of integers `stones` where `stones[i]` is the weight of the *i*th stone.

We are playing a game with the stones. On each turn, we choose the heaviest two stones and smash them together. Suppose the heaviest two stones have weights x and y with $x \leq y$. The result of this smash is:

- If $x == y$, both stones are destroyed, and
- If $x \neq y$, the stone of weight x is destroyed, and the stone of weight y has new weight $y - x$.

At the end of the game, there is at most one stone left.

Return the weight of the last remaining stone. If there are no stones left, return 0.

Example 1:

Input: stones = [2,7,4,1,8,1]

Output: 1

Explanation:

We combine 7 and 8 to get 1 so the array converts to [2,4,1,1,1] then, we combine 2 and 4 to get 2 so the array converts to [2,1,1,1] then, we combine 2 and 1 to get 1 so the array converts to [1,1,1] then, we combine 1 and 1 to get 0 so the array converts to [1] then that's the value of the last stone.

Example 2:

Input: stones = [1]

Output: 1

Constraints:

- $1 \leq \text{stones.length} \leq 30$
- $1 \leq \text{stones}[i] \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def lastStoneWeight(self, stones: list[int]) -> int:
        pq = [-stone for stone in stones]
        heapq.heapify(pq)

        while len(pq) >= 2:
            n1 = -heapq.heappop(pq)
            n2 = -heapq.heappop(pq)
            if n1 != n2:
                heapq.heappush(pq, -(n1 - n2))

        return 0 if not pq else -pq[0]
```

• LeetDoocs

```
class Solution:
    def lastStoneWeight(self, stones: List[int]) -> int:
        h = [-x for x in stones]
        heapify(h)
        while len(h) > 1:
            y, x = -heappop(h), -heappop(h)
            if x != y:
                heappush(h, x - y)
        return 0 if not h else -h[0]
```

• SimplyLeet

```
import heapq

def lastStoneWeight(stones):
    # Create a max heap by inserting negative values of stones
    max_heap = [-stone for stone in stones]
    heapq.heapify(max_heap)

    # While more than one stone remains
    while len(max_heap) > 1:
        # Get the heaviest stone (largest value, but stored as negative)

        heaviest = -heapq.heappop(max_heap)
        # Get the next heaviest stone
        next_heaviest = -heapq.heappop(max_heap)

        # If the stones are not equal, push the difference back into the heap
        if heaviest != next_heaviest:
            diff = heaviest - next_heaviest
            heapq.heappush(max_heap, -diff)

        # If there is a stone left, return its weight. Otherwise, return 0.

    return -max_heap[0] if max_heap else 0

# Example usage:
# print(lastStoneWeight([2,7,4,1,8,1]))
```

◆ Quick Review

Key Insights

- Use a max-heap (or simulate one) to always access the two largest stones quickly.
- If the two stones are equal, both are removed; otherwise, the difference is pushed back into the heap.
- Continue the process until the heap has one or no stone left.
- For languages without a max-heap in their standard library (e.g., Python), simulate one by pushing negative values.

Space & Time

Time Complexity: $O(n \log n)$ – Each stone removal and insertion takes $O(\log n)$ and in the worst-case scenario, we perform $O(n)$ operations.

Space Complexity: $O(n)$ – The heap stores all the stones initially.

Solution

We solve the problem by using a priority queue (max-heap). In Python, we simulate a max-heap by inserting the negative of each stone's weight into a min-heap. In JavaScript, due to the lack of a built-in heap structure, we

sort the array in descending order at each iteration (acceptable given the small constraints). In C++ and Java, we use the standard `priority_queue`. The algorithm repeatedly extracts the two largest stones, computes their difference, and if the difference is non-zero, it is inserted back into the heap. This process repeats until there is at most one stone left.

Round 5 · Mid · Medium

Round 5

Mid Priority · Medium

10 questions · 5 patterns

- ☐ ● **Linked List #138** ↗
- ☐ ● **Stack #853** ↗
- ☐ ● **Heap / Priority Queue #355** ↗
- ☐ ● **Backtracking #40 #131** ↗
- ☐ ● **Greedy #45 #678 #763 #846 #1899** ↗

Linked List

Round 5 · Mid · Medium · 1 question

Linked List

□ #138 Copy List with Random Pointer

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Linked List

Lists: NeetCode 150

Problem

A linked list of length n is given such that each node contains an additional random pointer, which could point to any node in the list, or null. Construct a deep copy of the list. The deep copy should consist of exactly n brand new nodes, where each new node has its value set to the value of its corresponding original node. Both the next and random pointer of the new nodes should point to new nodes in the copied list such that the pointers in the original list and copied list represent the same list state. None of the pointers in the new list should point to nodes in the original list.

For example, if there are two nodes X and Y in the original list, where $X.random \rightarrow Y$, then for the corresponding two nodes x and y in the

copied list, $x.random \rightarrow y$.

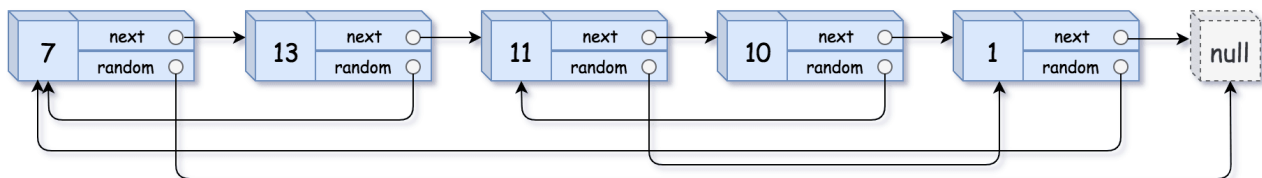
Return the head of the copied linked list.

The linked list is represented in the input/output as a list of n nodes. Each node is represented as a pair of $[val, random_index]$ where:

- **val**: an integer representing `Node.val`
- **random_index**: the index of the node (range from 0 to $n-1$) that the random pointer points to, or null if it does not point to any node.

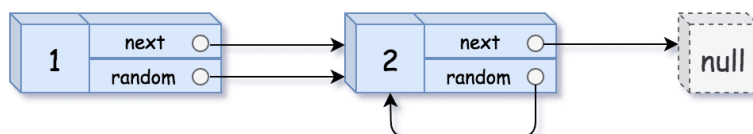
Your code will only be given the head of the original linked list.

Example 1:



Input: head = [[7,null],[13,0],[11,4],[10,2],[1,0]]
Output: [[7,null],[13,0],[11,4],[10,2],[1,0]]

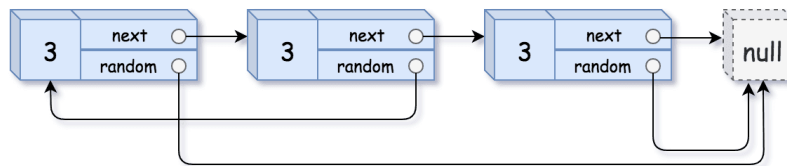
Example 2:



Input: head = [[1,1],[2,1]]

Output: [[1,1],[2,1]]

Example 3:



Input: head = [[3,null],[3,0],[3,null]]

Output: [[3,null],[3,0],[3,null]]

Constraints:

- $0 \leq n \leq 1000$
- $-104 \leq \text{Node.val} \leq 104$
- `Node.random` is null or is pointing to some node in the linked list.

Community Solutions (Python)

• WalkCC

```

class Solution:
    def copyRandomList(self, head: 'Node') -> 'Node':
        if not head:
            return None
        if head in self.map:
            return self.map[head]

        newNode = Node(head.val)
        self.map[head] = newNode
        newNode.next = self.copyRandomList(head.next)
  
```

```

newNode.random = self.copyRandomList(head.random)
return newNode

map = {}

```

• LeetCode

```

"""
# Definition for a Node.
class Node:
    def __init__(self, x: int, next: 'Node' = None, random: 'Node' = None):
        self.val = int(x)
        self.next = next
        self.random = random
"""

```

```

class Solution:
    def copyRandomList(self, head: "Optional[Node]") -> "Optional[Node]":
        d = {}
        dummy = tail = Node(0)
        cur = head
        while cur:
            node = Node(cur.val)
            tail.next = node
            tail = tail.next
            d[cur] = node

            cur = cur.next
        cur = head
        while cur:
            d[cur].random = d[cur.random] if cur.random else None
            cur = cur.next
        return dummy.next

```

• SimplyLeet

```

# Definition for a Node.
class Node:
    def __init__(self, val, next=None, random=None):
        self.val = val        # value of the node.
        self.next = next      # pointer to the next node.
        self.random = random  # pointer to a random node.

class Solution:
    def copyRandomList(self, head: 'Node') -> 'Node':
        # Return None if the head is None.

        if not head:
            return None

        # Dictionary to hold old node as key and new node as value.
        old_to_new = {}

        # First pass: create a copy of each node.
        current = head
        while current:
            old_to_new[current] = Node(current.val)

            current = current.next

        # Second pass: assign next and random pointers for the copied nodes.
        current = head
        while current:
            # Using dict.get ensures proper assignment or returns None.
            old_to_new[current].next = old_to_new.get(current.next)
            old_to_new[current].random = old_to_new.get(current.random)
            current = current.next

        # Return the head of the new copied linked list.
        return old_to_new[head]

```

◆ Quick Review

Key Insights

- Use a hash map (or dictionary) to map each original node to its corresponding copy.
- Perform two passes: the first to create a copy of each node, and the second to assign the next and random pointers using the hash map.
- An alternative interleaving approach exists but using a hash table is typically more straightforward.

Space & Time

Time Complexity: $O(n)$ – Each node is processed a constant number of times.

Space Complexity: $O(n)$ – A hash map is used to store the mapping from original nodes to their copies.

Solution

The solution uses a two-pass algorithm. In the first pass, we iterate through the original list and for each node, create a new node (deep copy) while saving the mapping between the original and copy in a hash table. In the second pass, we iterate over the original list again to assign the next and random pointers of the new nodes using the previously built map. This guarantees that every pointer in the

copied list corresponds to a new node and not the original.

Stack

Round 5 · Mid · Medium · 1 question

Stack

□ #853 Car Fleet

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack · Sorting · Monotonic Stack
Lists: NeetCode 150

Problem

There are n cars at given miles away from the starting mile 0, traveling to reach the mile target.

You are given two integer arrays `position` and `speed`, both of length n , where `position[i]` is the starting mile of the i th car and `speed[i]` is the speed of the i th car in miles per hour.

A car cannot pass another car, but it can catch up and then travel next to it at the speed of the slower car.

A car fleet is a single car or a group of cars driving next to each other. The speed of the car fleet is the minimum speed of any car in the fleet.

If a car catches up to a car fleet at the mile target, it will still be considered as part of the car fleet.

Return the number of car fleets that will arrive at the destination.

Example 1:

Input: target = 12, position = [10,8,0,5,3], speed = [2,4,1,1,3]

Output: 3

Explanation:

- The cars starting at 10 (speed 2) and 8 (speed 4) become a fleet, meeting each other at 12. The fleet forms at target.
- The car starting at 0 (speed 1) does not catch up to any other car, so it is a fleet by itself.
- The cars starting at 5 (speed 1) and 3 (speed 3) become a fleet, meeting each other at 6. The fleet moves at speed 1 until it reaches target.

Example 2:

Input: target = 10, position = [3], speed = [3]

Output: 1

Explanation:

Example 3:

Input: target = 100, position = [0,2,4], speed = [4,2,1]

Output: 1

Explanation:

- The cars starting at 0 (speed 4) and 2 (speed 2) become a fleet, meeting each other at 4. The car starting at 4 (speed 1) travels to 5.
- Then, the fleet at 4 (speed 2) and the car at position 5 (speed 1) become one fleet, meeting each other at 6. The fleet moves at speed 1 until it reaches target.

Constraints:

- $n == \text{position.length} == \text{speed.length}$
- $1 \leq n \leq 105$
- $0 < \text{target} \leq 10^6$
- $0 \leq \text{position}[i] < \text{target}$
- All the values of position are unique.
- $0 < \text{speed}[i] \leq 10^6$

Community Solutions (Python)

- WalkCC

```

class Solution:
    def carFleet(self, target: int, position: list[int], speed: list[int]) -> int:
        ans = 0
        times = [
            float(target - p) / s for p, s in sorted(zip(position, speed),
                                                    reverse=True)]

        maxTime = 0 # the time of the slowest car to reach the target

        for time in times:
            # A car needs more time to reach the target, so it becomes the slowest.

            if time > maxTime:
                maxTime = time
                ans += 1

        return ans

```

• LeetCode

```

class Solution:
    def carFleet(self, target: int, position: List[int], speed: List[int]) -> int:
        idx = sorted(range(len(position)), key=lambda i: position[i])
        ans = pre = 0
        for i in idx[::-1]:
            t = (target - position[i]) / speed[i]
            if t > pre:
                ans += 1
                pre = t
        return ans

```

• SimplyLeet

```
# Python implementation of Car Fleet
def carFleet(target, position, speed):
    # Pair up position and speed and sort by starting position in descending
    # order.
    cars = sorted(zip(position, speed), key=lambda car: car[0], reverse=True)

    fleets = 0          # Count of car fleets
    last_time = 0.0     # Time taken by the last fleet to reach the target

    # Process each car from closest to farthest from the target.
    for pos, spd in cars:

        # Calculate time for this car to reach the target
        time = (target - pos) / spd

        # If this car takes longer than the last fleet time, it starts a new
        # fleet.
        if time > last_time:
            fleets += 1
            last_time = time # Update the last fleet time with this car's time

    return fleets

# Example usage:
# target = 12, position = [10,8,0,5,3], speed = [2,4,1,1,3]
# print(carFleet(12, [10,8,0,5,3], [2,4,1,1,3])) # Output: 3
```

◆ Quick Review

Key Insights

- Since cars cannot pass each other, the relative order (by starting position) is maintained.
- Sorting the cars based on their starting positions (descending order) helps to simulate

the catching-up process.

- Computing the time each car takes to reach the target is critical.
- Compare the current car's time to the time of the fleet ahead: if it takes longer, it forms a new fleet; otherwise, it gets merged.
- A stack-like approach (tracking the slowest arrival time seen so far) reduces the problem to a single pass after sorting.

Space & Time

Time Complexity: $O(n \log n)$ due to the sorting step.

Space Complexity: $O(n)$ for storing the computed times and sorted car information.

Solution

The solution involves sorting the cars by their starting positions in descending order so that we process the cars from nearest to the target to farthest. For each car, calculate the time to reach the target using the formula $(\text{target} - \text{position}) / \text{speed}$. As we iterate:

- If a car's time is greater than the slowest time encountered (tracked by a variable or stack), it cannot catch up to the fleet ahead

and forms a new fleet. – Otherwise, it merges into the current fleet. This approach ensures that we count each distinct fleet accurately with a single pass through the sorted list.

Heap / Priority Queue

Round 5 · Mid · Medium · 1 question

Heap / Priority Queue

□ #355 Design Twitter

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Linked List · Design · Heap
(Priority Queue)

Lists: NeetCode 150

Problem

Design a simplified version of Twitter where users can post tweets, follow/unfollow another user, and is able to see the 10 most recent tweets in the user's news feed.

Implement the Twitter class:

- **Twitter()** Initializes your twitter object.
- **void postTweet(int userId, int tweetId)**

Composes a new tweet with ID tweetId by the user userId. Each call to this function will be made with a unique tweetId.

- **List<Integer> getNewsFeed(int userId)**

Retrieves the 10 most recent tweet IDs in the user's news feed. Each item in the news feed must be posted by users who the user

followed or by the user themselves. Tweets must be ordered from most recent to least recent.

- `void follow(int followerId, int followeId)`

The user with ID `followerId` started following the user with ID `followeId`.

- `void unfollow(int followerId, int followeId)`

The user with ID `followerId` started unfollowing the user with ID `followeId`.

Example 1:

Input

```
["Twitter", "postTweet", "getNewsFeed", "follow", "postTweet", "getNewsFeed",  
"unfollow", "getNewsFeed"]
```

```
[[], [1, 5], [1], [1, 2], [2, 6], [1], [1, 2], [1]]
```

Output

```
[null, null, [5], null, null, [6, 5], null, [5]]
```

Explanation

```
Twitter twitter = new Twitter();
```

Constraints:

- $1 \leq \text{userId}, \text{followerId}, \text{followeId} \leq 500$
- $0 \leq \text{tweetId} \leq 10^4$
- All the tweets have unique IDs.
- At most $3 * 10^4$ calls will be made to `postTweet`, `getNewsFeed`, `follow`, and `unfollow`.
- A user cannot follow himself.

Community Solutions (Python)

• WalkCC

```
class Twitter:
    def __init__(self):
        self.timer = itertools.count(step=-1)
        self.tweets = collections.defaultdict(deque)
        self.followees = collections.defaultdict(set)

    def postTweet(self, userId: int, tweetId: int) -> None:
        self.tweets[userId].appendleft((next(self.timer), tweetId))
        if len(self.tweets[userId]) > 10:
            self.tweets[userId].pop()

    def getNewsFeed(self, userId: int) -> list[int]:
        tweets = list(
            heapq.merge(
                *
                (self.tweets[followee]
                 for followee in self.followees[userId] | {userId}))
        )
        return [tweetId for _, tweetId in tweets[:10]]

    def follow(self, followerId: int, followeeId: int) -> None:
        self.followees[followerId].add(followeeId)

    def unfollow(self, followerId: int, followeeId: int) -> None:
        self.followees[followerId].discard(followeeId)
```

• LeetDoocs

```
class Twitter:
    def __init__(self):
        """
        Initialize your data structure here.
        """
        self.user_tweets = defaultdict(list)
        self.user_following = defaultdict(set)
        self.tweets = defaultdict()
        self.time = 0
```

```
def postTweet(self, userId: int, tweetId: int) -> None:
    """
    Compose a new tweet.
    """
    self.time += 1
    self.user_tweets[userId].append(tweetId)
    self.tweets[tweetId] = self.time

def getNewsFeed(self, userId: int) -> List[int]:
    """
```

Retrieve the 10 most recent tweet ids in the user's news feed. Each item in the news feed must be posted by users who the user followed or by the user herself. Tweets must be ordered from most recent to least recent.

```
    """
    following = self.user_following[userId]
    users = set(following)
    users.add(userId)
    tweets = [self.user_tweets[u][::-1][:10] for u in users]
    tweets = sum(tweets, [])
    return nlargest(10, tweets, key=lambda tweet: self.tweets[tweet])

def follow(self, followerId: int, followeeId: int) -> None:
```

```
    """
    Follower follows a followee. If the operation is invalid, it should be
    a no-op.
    """
    self.user_following[followerId].add(followeeId)

def unfollow(self, followerId: int, followeeId: int) -> None:
    """
    Follower unfollows a followee. If the operation is invalid, it should
    be a no-op.
    """
    following = self.user_following[followerId]
```

```

    if followeeId in following:
        following.remove(followeeId)

```

```

# Your Twitter object will be instantiated and called as such:
# obj = Twitter()
# obj.postTweet(userId,tweetId)
# param_2 = obj.getNewsFeed(userId)
# obj.follow(followerId,followeeId)
# obj.unfollow(followerId,followeeId)

```

• SimplyLeet

```

# Python implementation of the Twitter design problem
import heapq

class Twitter:
    def __init__(self):
        # Mapping from userId to a set of followeeIds
        self.follows = {}
        # Mapping from userId to list of (timestamp, tweetId)
        self.tweets = {}
        # Global timestamp to order tweets

        self.timestamp = 0

    def postTweet(self, userId: int, tweetId: int) -> None:
        # Ensure the user follows itself for easier news feed retrieval
        if userId not in self.follows:
            self.follows[userId] = set()
        self.follows[userId].add(userId)

        # Append the tweet with the current timestamp, then increment timestamp
        if userId not in self.tweets:

```

```

        self.tweets[userId] = []
        self.tweets[userId].append((self.timestamp, tweetId))
        self.timestamp += 1

def getNewsFeed(self, userId: int) -> list:
    # If the user does not exist in follows mapping, return empty feed
    if userId not in self.follows:
        return []

    # Use a heap (as a max-heap using negative timestamp)

    heap = []
    # Iterate over all followees of the user
    for followeeId in self.follows[userId]:
        if followeeId in self.tweets and self.tweets[followeeId]:
            # Start from the most recent tweet of the followee
            index = len(self.tweets[followeeId]) - 1
            ts, tid = self.tweets[followeeId][index]
            # Push the tweet along with followeeId and the index of the
            tweet in the list
            heapq.heappush(heap, (-ts, tid, followeeId, index))

    result = []
    # Retrieve up to 10 tweets
    while heap and len(result) < 10:
        neg_ts, tweetId, followeeId, idx = heapq.heappop(heap)
        result.append(tweetId)
        # If there is an older tweet from this followee, push it onto the
        heap

        if idx - 1 >= 0:
            ts, tid = self.tweets[followeeId][idx - 1]
            heapq.heappush(heap, (-ts, tid, followeeId, idx - 1))
    return result

```

```
def follow(self, followerId: int, followeeId: int) -> None:
    # A user should always follow itself
    if followerId not in self.follows:
        self.follows[followerId] = set()
    self.follows[followerId].add(followerId)
    # Add followeeId to follower's follow set if not self-follow
    if followerId != followeeId:
        self.follows[followerId].add(followeeId)

def unfollow(self, followerId: int, followeeId: int) -> None:
    # A user cannot unfollow itself
    if followerId in self.follows and followeeId != followerId:
        self.follows[followerId].discard(followeeId)
```

◆ Quick Review

Key Insights

- Use a global timestamp to order tweets as they are posted.
- Maintain a mapping of each user to the list of tweets they posted.
- Maintain a mapping of each user to the set of users they follow.
- For getNewsFeed, merge tweets from the user and their followees using a heap to efficiently retrieve the 10 most recent tweets.
- Ensure that a user always follows themselves to simplify retrieving their own tweets.

Space & Time

Time Complexity:

- **postTweet: $O(1)$**
- **follow / unfollow: $O(1)$**
- **getNewsFeed: $O(N \log K)$ where N is the number of followees and K is the average number of tweets considered (capped at 10 tweets per merge iteration)**

Space Complexity:

- **$O(U + T)$ where U is the number of users (for follow mappings) and T is the total number of tweets stored.**

Solution

We use two main data structures: a hash table to store tweets for each user and another hash table for follow relationships. Each tweet is stored as a tuple of (timestamp, tweetId). A global timestamp counter provides a unique ordering. A min-heap (or max-heap using negative timestamp values) is used to merge the tweets from different users efficiently.

When retrieving the news feed, we look up the tweet lists for the given user's followees (including self) and use the heap to extract the 10 most recent tweets.

Backtracking

Round 5 · Mid · Medium · 2 questions

Backtracking

□ #40 Combination Sum II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Backtracking
Lists: NeetCode 150

Problem

Given a collection of candidate numbers (candidates) and a target number (target), find all unique combinations in candidates where the candidate numbers sum to target.

Each number in candidates may only be used once in the combination.

Note: The solution set must not contain duplicate combinations.

Example 1:

```
Input: candidates = [10,1,2,7,6,1,5], target = 8
Output:
[
  [1,1,6],
  [1,2,5],
  [1,7],
  [2,6]
]
```

Example 2:

Input: candidates = [2,5,2,1,2], target = 5

Output:

```
[  
  [1,2,2],  
  [5]  
]
```

Constraints:

- $1 \leq \text{candidates.length} \leq 100$
- $1 \leq \text{candidates}[i] \leq 50$
- $1 \leq \text{target} \leq 30$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def combinationSum2(self, candidates: list[int],
                       target: int) -> list[list[int]]:

        ans = []

        def dfs(s: int, target: int, path: list[int]) -> None:
            if target < 0:
                return
            if target == 0:
                ans.append(path.copy())

            return

        for i in range(s, len(candidates)):
            if i > s and candidates[i] == candidates[i - 1]:
                continue
            path.append(candidates[i])
            dfs(i + 1, target - candidates[i], path)
            path.pop()

        candidates.sort()

        dfs(0, target, [])
        return ans
```

• LeetCode

```
class Solution:
    def combinationSum2(self, candidates: List[int], target: int) -> List[List[int]]:
        def dfs(i: int, s: int):
            if s == 0:
                ans.append(t[:])
                return
            if i >= len(candidates) or s < candidates[i]:
                return
            for j in range(i, len(candidates)):
                if j > i and candidates[j] == candidates[j - 1]:
                    continue
                t.append(candidates[j])
                dfs(j + 1, s - candidates[j])
                t.pop()

        candidates.sort()
        ans = []
        t = []
        dfs(0, target)
        return ans
```

• SimplyLeet

```
# Python Solution for Combination Sum II

def combinationSum2(candidates, target):
    # Sort candidates to handle duplicates easily.
    candidates.sort()
    results = []

    def backtrack(start, current_combination, current_sum):
        # If the current sum equals target, add a copy of the current
        # combination to results.
        if current_sum == target:
```

```
        results.append(list(current_combination))
        return
    # Iterate over possible candidates starting from index 'start'.
    for i in range(start, len(candidates)):
        # Skip duplicate elements.
        if i > start and candidates[i] == candidates[i - 1]:
            continue
        new_sum = current_sum + candidates[i]
        # If the new sum exceeds target, break the loop.
        if new_sum > target:
            break
        # Include the candidate and recurse.
        current_combination.append(candidates[i])
        backtrack(i + 1, current_combination, new_sum)
        # Backtrack: remove the last element added.
        current_combination.pop()

    backtrack(0, [], 0)
    return results
```

```
# Example usage:
print(combinationSum2([10,1,2,7,6,1,5], 8))
```

◆ Quick Review

Key Insights

- Sort the candidates first to easily skip duplicates.
- Use backtracking to explore all possible combinations.
- Avoid selecting the same number (at the same recursive level) to avoid duplicate combinations.

- Terminate recursion when the running sum exceeds the target.
- Use indices rather than reusing elements to ensure each number is used at most once.

Space & Time

Time Complexity: $O(2^n * k)$ in the worst case, where n is the number of candidates and k is the average length of a combination.

Space Complexity: $O(k)$ for the recursion stack and current combination (not including the space occupied by the output).

Solution

The solution is based on a backtracking algorithm. First, the candidates are sorted to handle duplicates easily. Then, a recursive function is used to build combinations. At each recursion level, iterate through the candidates starting from the current index. For each candidate:

- Skip the candidate if it is the same as the previous one (to avoid duplicates).
- If the candidate can be added without exceeding the target, add it to the current combination.
- Recurse with the next index (ensuring each

candidate is used only once). – Backtrack by removing the candidate from the current combination once completed.

Data structures used include:

– An array or list to hold the current combination. – A results list to store all valid combinations. – Sorting is leveraged to simplify duplicate elimination.

Backtracking

□ #131 Palindrome Partitioning

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming · Backtracking
Lists: NeetCode 150

Problem

Given a string *s*, partition *s* such that every substring of the partition is a palindrome.
Return all possible palindrome partitioning of *s*.

Example 1:

```
Input: s = "aab"  
Output: [["a","a","b"],["aa","b"]]
```

Example 2:

```
Input: s = "a"  
Output: [["a"]]
```

Constraints:

- $1 \leq s.length \leq 16$
- *s* contains only lowercase English letters.

Community Solutions (Python)

- • WalkCC

```

class Solution:
    def partition(self, s: str) -> list[list[str]]:
        ans = []

        def isPalindrome(s: str) -> bool:
            return s == s[::-1]

        def dfs(s: str, j: int, path: list[str], ans: list[list[str]]) -> None:
            if j == len(s):
                ans.append(path)

            return

            for i in range(j, len(s)):
                if isPalindrome(s[j: i + 1]):
                    dfs(s, i + 1, path + [s[j: i + 1]], ans)

        dfs(s, 0, [], ans)
        return ans

```

• LeetDoocs

```

class Solution:
    def partition(self, s: str) -> List[List[str]]:
        def dfs(i: int):
            if i == n:
                ans.append(t[:])
                return
            for j in range(i, n):
                if f[i][j]:
                    t.append(s[i : j + 1])
                    dfs(j + 1)

            t.pop()

        n = len(s)
        f = [[True] * n for _ in range(n)]
        for i in range(n - 1, -1, -1):
            for j in range(i + 1, n):
                f[i][j] = s[i] == s[j] and f[i + 1][j - 1]
        ans = []
        t = []
        dfs(0)

```

```
return ans
```

• SimplyLeet

```
def partition(s):
    # Helper function to check if a given substring is a palindrome
    def is_palindrome(sub):
        return sub == sub[::-1]

    result = [] # final list to store all possible partitions

    # backtracking helper function: current_index points to the starting index
    # of the current search,
    # current_partition stores the partition done so far.
    def backtrack(current_index, current_partition):

        # if we reached the end of the string, add the current_partition to the
        # result list
        if current_index == len(s):
            result.append(list(current_partition))
            return

        # try all possible substrings starting from current_index
        for i in range(current_index, len(s)):
            substring = s[current_index:i+1] # current substring
            # if the substring is a palindrome, add it to current_partition
            if is_palindrome(substring):
                current_partition.append(substring)

                backtrack(i+1, current_partition)
                # backtrack: remove the last added substring
                current_partition.pop()

        backtrack(0, [])
    return result

# Example usage:
print(partition("aab"))
```

◆ Quick Review

Key Insights

- Use backtracking to explore all possible ways to partition the string.
- At each step, check whether the current substring is a palindrome.
- If it is, include that substring in the current partition and recursively partition the rest of the string.
- The limit on the string length (at most 16 characters) allows an exponential backtracking approach.
- Optimizations like caching palindrome checks can be used, though the simple approach is typically sufficient.

Space & Time

Time Complexity: $O(n * 2^n)$ in the worst case, due to exploring all possible partitions and checking palindromes.

Space Complexity: $O(n)$ for recursion stack and temporary storage for current partition.

Solution

The solution uses the backtracking algorithm:

- Define a helper function that will recursively try to partition the string.
- For each possible

split, check if the substring is a palindrome. – If it is, add the substring to the current partition list and recursively call the helper on the remaining substring. – When the end of the string is reached, add the current partition list to the final result. – Backtrack by removing the last added substring and try the next possible partition. This method systematically explores every possible palindrome partitioning of the string.

Greedy

Round 5 · Mid · Medium · 5 questions

Greedy

□ #45 Jump Game II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Greedy
Lists: NeetCode 150

Problem

You are given a 0-indexed array of integers `nums` of length `n`. You are initially positioned at index 0.

Each element `nums[i]` represents the maximum length of a forward jump from index `i`. In other words, if you are at index `i`, you can jump to any index $(i + j)$ where:

- $0 \leq j \leq \text{nums}[i]$ and
- $i + j < n$

Return the minimum number of jumps to reach index `n - 1`. The test cases are generated such that you can reach index `n - 1`.

Example 1:

Input: `nums = [2,3,1,1,4]`

Output: 2

Explanation: The minimum number of jumps to reach the last index is 2. Jump 1 step from index 0 to 1, then 3 steps to the last index.

Example 2:

Input: nums = [2,3,0,1,4]
Output: 2

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $0 \leq \text{nums}[i] \leq 1000$
- It's guaranteed that you can reach $\text{nums}[n - 1]$.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def jump(self, nums: list[int]) -> int:
        ans = 0
        end = 0
        farthest = 0

        # Start an implicit BFS.
        for i in range(len(nums) - 1):
            farthest = max(farthest, i + nums[i])
            if farthest >= len(nums) - 1:

                ans += 1
                break
            if i == end:      # Visited all the items on the current level.
                ans += 1     # Increment the level.
                end = farthest # Make the queue size for the next level.

        return ans
```

• LeetDoocs

```
class Solution:
    def jump(self, nums: List[int]) -> int:
        ans = mx = last = 0
        for i, x in enumerate(nums[:-1]):
            mx = max(mx, i + x)
            if last == i:
                ans += 1
                last = mx
        return ans
```

• SimplyLeet

```
# Function to calculate minimum number of jumps to reach the last index
def jump(nums):
    # If the list has only one element, we're already at the destination.
    if len(nums) == 1:
        return 0
```

```
    n = len(nums) # Length of the input array
    jumps = 0     # To count the number of jumps required
    currentEnd = 0 # The end of the current range that can be reached
    furthest = 0  # The furthest index that can be reached from the current
    range
```

```
    # Iterate till the second last element because we don't need to jump from
    the last element
    for i in range(n - 1):
        # Update the furthest index reachable from index i
        furthest = max(furthest, i + nums[i])
        # If we have reached the end of the current jump range
        if i == currentEnd:
            jumps += 1 # Increase the jump count as we make a jump
            currentEnd = furthest # Update the currentEnd to the furthest
    reachable index
    return jumps
```

```
# Example usage:
print(jump([2,3,1,1,4])) # Output: 2
```

◆ Quick Review

Key Insights

- Utilize a greedy algorithm to make optimal jumps.
- Track the furthest index that can be reached within the current jump range.
- When the current index reaches the end of the current range, a jump is made and the range updates to the furthest reachable index.
- Edge case: if the array has only one element, no jumps are needed.

Space & Time

Time Complexity: $O(n)$ – Single pass through the array.

Space Complexity: $O(1)$ – Only a few variables are used for tracking jumps and ranges.

Solution

We use a greedy approach by iterating through the array and keeping track of:

- The furthest index reachable from the current range (furthest).
- The end of the current jump's range (currentEnd).
- The number of jumps made (jumps).

At each iteration, we update the furthest reachable index. When the current index reaches the currentEnd, it indicates that one jump has been used and we update currentEnd to furthest. This ensures we are always making the optimal jump to cover the longest distance possible until the end of the array is reached.

Greedy

□ #678 Valid Parenthesis String

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming · Stack · Greedy
Lists: NeetCode 150

Problem

Given a string *s* containing only three types of characters: '(', ')' and '*', return true if *s* is valid.

The following rules define a valid string:

- Any left parenthesis '(' must have a corresponding right parenthesis ')'.
• Any right parenthesis ')' must have a corresponding left parenthesis '('.
- Left parenthesis '(' must go before the corresponding right parenthesis ')'.
• '*' could be treated as a single right parenthesis ')' or a single left parenthesis '(' or an empty string "".

Example 1:

```
Input: s = "()"
Output: true
```

Example 2:

Input: s = "(*)"
Output: true

Example 3:

Input: s = "(*))"
Output: true

Constraints:

- $1 \leq s.length \leq 100$
- s[i] is '(', ')' or '*'.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def checkValidString(self, s: str) -> bool:
        low = 0
        high = 0

        for c in s:
            if c == '(':
                low += 1
                high += 1
            elif c == ')':
                if low > 0:
                    low -= 1
                high -= 1
            else:
                if low > 0:
                    low -= 1
                high += 1
            if high < 0:
                return False
```

```
return low == 0
```

• LeetDoocs

```
class Solution:
    def checkValidString(self, s: str) -> bool:
        n = len(s)
        dp = [[False] * n for _ in range(n)]
        for i, c in enumerate(s):
            dp[i][i] = c == '*'
        for i in range(n - 2, -1, -1):
            for j in range(i + 1, n):
                dp[i][j] = (
                    s[i] in '(*' and s[j] in '*)' and (i + 1 == j or dp[i + 1]
[j - 1])
                )
                dp[i][j] = dp[i][j] or any(
                    dp[i][k] and dp[k + 1][j] for k in range(i, j)
                )
        return dp[0][-1]
```

```
class Solution:
    def checkValidString(self, s: str) -> bool:
        x = 0
        for c in s:
            if c in '(':
                x += 1
            elif x:
                x -= 1
            else:
                return False
```

```
        x = 0
        for c in s[::-1]:
            if c in ')':
                x += 1
            elif x:
                x -= 1
            else:
                return False
        return True
```

• SimplyLeet

```
# Python implementation with detailed inline comments
```

```
def checkValidString(s):  
    # Initialize the minimum and maximum possible count of open parentheses  
    minOpen = 0  
    maxOpen = 0
```

```
    # Iterate over each character in the string
```

```
    for char in s:  
        if char == '(':
```

```
            # For an open parenthesis, both min and max increase by 1
```

```
            minOpen += 1
```

```
            maxOpen += 1
```

```
        elif char == ')':
```

```
            # For a close parenthesis, both min and max decrease by 1
```

```
            minOpen -= 1
```

```
            maxOpen -= 1
```

```
        else: # char == '*'
```

```
            # '*' can act as a ')' (decrease min) or '(' (increase max)
```

```
            minOpen -= 1
```

```
            maxOpen += 1
```

```
    # Ensure minOpen does not drop below 0
```

```
    if minOpen < 0:
```

```
        minOpen = 0
```

```
    # If maxOpen is negative, there are too many ')' to balance
```

```
    if maxOpen < 0:
```

```
        return False
```

```
    # The string is valid if there is a possible balance where minOpen equals 0
```

```
    return minOpen == 0
```

```
# Example usage:
```

```
print(checkValidString("(*))") # Expected output: True
```

◆ Quick Review

Key Insights

- The '*' character provides flexibility, allowing multiple interpretations.
- A greedy approach can track the range of possible open parentheses count.
- Use two counters to represent the minimum and maximum open count possible at any moment.
- If the maximum count goes negative at any point, the string cannot be valid.
- At the end, the minimum count must be zero for a valid string.

Space & Time

Time Complexity: $O(n)$ where n is the length of the string

Space Complexity: $O(1)$

Solution

We use a greedy algorithm by maintaining two counters:

- minOpen – the minimum number of open parentheses that could be present.
- maxOpen – the maximum number of open parentheses that could be present.

For every character in the string:

– For '(', increment both minOpen and maxOpen. – For ')', decrement both minOpen and maxOpen. – For '*', decrement minOpen (treat as ')') and increment maxOpen (treat as '(').

Clamp minOpen to not go below 0, since negative count of open parentheses is not possible in any valid interpretation. If maxOpen drops below 0 at any point, it means there are more closing brackets than can be balanced by available '*' and '(', so the string is invalid. Finally, if minOpen is 0, the string can be balanced, otherwise it is invalid.

Greedy

□ #763 Partition Labels

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Two Pointers · String · Greedy
Lists: NeetCode 150

Problem

You are given a string *s*. We want to partition the string into as many parts as possible so that each letter appears in at most one part. For example, the string "ababcc" can be partitioned into ["abab", "cc"], but partitions such as ["aba", "bcc"] or ["ab", "ab", "cc"] are invalid.

Note that the partition is done so that after concatenating all the parts in order, the resultant string should be *s*.

Return a list of integers representing the size of these parts.

Example 1:

Input: s = "ababcbacacdefegdehijklij"

Output: [9,7,8]

Explanation:

The partition is "ababcbaca", "defegde", "hijklij".

This is a partition so that each letter appears in at most one part.

A partition like "ababcbacacdefegde", "hijklij" is incorrect, because it splits s into less parts.

Example 2:

Input: s = "eccbbbbbdec"

Output: [10]

Constraints:

- $1 \leq s.length \leq 500$
- s consists of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def partitionLabels(self, s: str) -> list[int]:
        ans = []
        letterToRightmostIndex = {c: i for i, c in enumerate(s)}

        l = 0 # the leftmost index of the current running string
        r = 0 # the rightmost index of the current running string

        for i, c in enumerate(s):
            r = max(r, letterToRightmostIndex[c])

            if i == r:
                ans.append(r - l + 1)
                l = r + 1

        return ans
```

• LeetDoocs

```

class Solution:
    def partitionLabels(self, s: str) -> List[int]:
        last = {c: i for i, c in enumerate(s)}
        mx = j = 0
        ans = []
        for i, c in enumerate(s):
            mx = max(mx, last[c])
            if mx == i:
                ans.append(i - j + 1)
                j = i + 1

        return ans

```

• SimplyLeet

```

# Python Solution
class Solution:
    def partitionLabels(self, s: str) -> list:
        # Step 1: Record the last occurrence of each character.
        last_occurrence = {char: i for i, char in enumerate(s)}
        # List to store the sizes of partitions.
        partitions = []
        start, end = 0, 0

        # Step 2: Iterate over the string to form partitions.

        for i, char in enumerate(s):
            # Update the end index for the current partition.
            end = max(end, last_occurrence[char])
            # When we reach the end of the partition.
            if i == end:
                partitions.append(end - start + 1)
                start = i + 1 # Move start to the next index.

        return partitions

```

◆ Quick Review

Key Insights

- Determine the last occurrence of every character in the string.
- Use a greedy approach to extend the current partition until all characters within it do not appear later in the string.
- When the current index reaches the furthest last occurrence for characters seen so far, a partition can be closed.
- Repeat the process for the remaining section of the string.

Space & Time

Time Complexity: $O(n)$ where n is the length of the string because we traverse the string twice.

Space Complexity: $O(1)$ (or $O(26)$ to be precise) for storing the last occurrences of each character.

Solution

We first iterate through the string to store the last index at which each character appears. Then, we traverse the string while maintaining two pointers: one for the start of the current partition and one to track the furthest last occurrence (end) of any character seen so far.

For each character, we update the end pointer to be the maximum of its current value and the last index of that character. When the current index equals the end pointer, it means that all characters in the current partition do not appear later. We then determine the size of this partition, add it to the result list, and move to the next partition. We repeat this until the entire string is processed.

Greedy

□ #846 Hand of Straights

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Greedy · Sorting
Lists: NeetCode 150

Problem

Alice has some number of cards and she wants to rearrange the cards into groups so that each group is of size `groupSize`, and consists of `groupSize` consecutive cards.

Given an integer array `hand` where `hand[i]` is the value written on the *i*th card and an integer `groupSize`, return `true` if she can rearrange the cards, or `false` otherwise.

Example 1:

Input: `hand = [1,2,3,6,2,3,4,7,8]`, `groupSize = 3`
Output: `true`
Explanation: Alice's hand can be rearranged as `[1,2,3]`, `[2,3,4]`, `[6,7,8]`

Example 2:

Input: `hand = [1,2,3,4,5]`, `groupSize = 4`
Output: `false`
Explanation: Alice's hand can not be rearranged into groups of 4.

Constraints:

- $1 \leq \text{hand.length} \leq 104$
- $0 \leq \text{hand}[i] \leq 109$
- $1 \leq \text{groupSize} \leq \text{hand.length}$

Note: This question is the same as 1296: <https://leetcode.com/problems/divide-array-in-sets-of-k-consecutive-numbers/>

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isNStraightHand(self, hand: list[int], groupSize: int) -> bool:
        count = collections.Counter(hand)

        for start in sorted(count):
            value = count[start]
            if value > 0:
                for i in range(start, start + groupSize):
                    count[i] -= value
                    if count[i] < 0:
                        return False

        return True
```

• LeetDoocs

```

class Solution:
    def isNStraightHand(self, hand: List[int], groupSize: int) -> bool:
        if len(hand) % groupSize:
            return False
        cnt = Counter(hand)
        for x in sorted(hand):
            if cnt[x]:
                for y in range(x, x + groupSize):
                    if cnt[y] == 0:
                        return False
                    cnt[y] -= 1
        return True

```

• SimplyLeet

```

# Python solution using collections.Counter for frequency mapping
from collections import Counter

def isNStraightHand(hand, groupSize):
    # Check whether total cards can be divided into groups of groupSize
    if len(hand) % groupSize != 0:
        return False

    # Create frequency map of cards
    card_count = Counter(hand)

    # Process sorted keys to ensure we always start with the smallest available
    # card
    for card in sorted(card_count.keys()):
        # If current count is greater than 0, try to form a consecutive group
        # starting from card
        count = card_count[card]
        if count > 0:
            for i in range(groupSize):
                # Check if consecutive card exists in sufficient quantity
                if card_count[card + i] < count:
                    return False

```

```
        # Deduct the frequency for the current group
        card_count[card + i] -= count

    return True

# Example usage:
# print(isNStraightHand([1,2,3,6,2,3,4,7,8], 3)) # Expected output: True
```

◆ Quick Review

Key Insights

- The total number of cards must be divisible by `groupSize`.
- Use a frequency count (hash table or map) to record the occurrence of each card.
- Always start forming groups from the smallest available card to maintain the consecutive order.
- For each card, if its count is non-zero, check that the subsequent `groupSize-1` cards exist with at least as many counts.
- Decrease the counts accordingly for each valid consecutive group.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting the unique keys from the hand.

Space Complexity: $O(n)$ for storing the frequency count of the cards.

Solution

The solution uses a greedy approach along with a frequency hash map. The main steps are:

– Verify that the total number of cards is divisible by groupSize. – Count the frequency of each card. – Sort the keys of the frequency map. – Iterate over the sorted keys and for each card with a positive count, try to form a group of groupSize consecutive cards. – For each card in the group, decrease the frequency by the number of groups formed. If at any point the required card is missing or its count is insufficient, return false. – If all groups can be formed successfully, return true.

Greedy

□ #1899 Merge Triplets to Form Target Triplet

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Greedy
Lists: NeetCode 150

Problem

A triplet is an array of three integers. You are given a 2D integer array triplets, where $\text{triplets}[i] = [a_i, b_i, c_i]$ describes the i th triplet. You are also given an integer array target = $[x, y, z]$ that describes the triplet you want to obtain.

To obtain target, you may apply the following operation on triplets any number of times (possibly zero):

- Choose two indices (0-indexed) i and j ($i \neq j$) and update $\text{triplets}[j]$ to become $[\max(a_i, a_j), \max(b_i, b_j), \max(c_i, c_j)]$. For example, if $\text{triplets}[i] = [2, 5, 3]$ and $\text{triplets}[j] = [1, 7, 5]$, $\text{triplets}[j]$ will be updated to $[\max(2, 1), \max(5, 7), \max(3, 5)] = [2, 7, 5]$.

Return true if it is possible to obtain the target triplet $[x, y, z]$ as an element of triplets, or false otherwise.

Example 1:

Input: triplets = $[[2,5,3],[1,8,4],[1,7,5]]$, target = $[2,7,5]$

Output: true

Explanation: Perform the following operations:

– Choose the first and last triplets $[[2,5,3],[1,8,4],[1,7,5]]$. Update the last triplet to be $[\max(2,1), \max(5,7), \max(3,5)] = [2,7,5]$. triplets = $[[2,5,3],[1,8,4],[2,7,5]]$

The target triplet $[2,7,5]$ is now an element of triplets.

Example 2:

Input: triplets = $[[3,4,5],[4,5,6]]$, target = $[3,2,5]$

Output: false

Explanation: It is impossible to have $[3,2,5]$ as an element because there is no 2 in any of the triplets.

Example 3:

Input: triplets = $[[2,5,3],[2,3,4],[1,2,5],[5,2,3]]$, target = $[5,5,5]$

Output: true

Explanation: Perform the following operations:

– Choose the first and third triplets $[[2,5,3],[2,3,4],[1,2,5],[5,2,3]]$. Update the third triplet to be $[\max(2,1), \max(5,2), \max(3,5)] = [2,5,5]$. triplets = $[[2,5,3],[2,3,4],[2,5,5],[5,2,3]]$.

– Choose the third and fourth triplets $[[2,5,3],[2,3,4],[2,5,5],[5,2,3]]$. Update the fourth triplet to be $[\max(2,5), \max(5,2), \max(5,3)] = [5,5,5]$. triplets = $[[2,5,3],[2,3,4],[2,5,5],[5,5,5]]$.

The target triplet $[5,5,5]$ is now an element of triplets.

Constraints:

- $1 \leq \text{triplets.length} \leq 105$
- $\text{triplets}[i].\text{length} == \text{target.length} == 3$
- $1 \leq a_i, b_i, c_i, x, y, z \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def mergeTriplets(self, triplets: list[list[int]], target: list[int]) -> bool:
        merged = [0] * len(target)

        for triplet in triplets:
            if all(a <= b for a, b in zip(triplet, target)):
                for i in range(3):
                    merged[i] = max(merged[i], triplet[i])

        return merged == target
```

• LeetDoocs

```
class Solution:
    def mergeTriplets(self, triplets: List[List[int]], target: List[int]) -> bool:
        x, y, z = target
        d = e = f = 0
        for a, b, c in triplets:
            if a <= x and b <= y and c <= z:
                d = max(d, a)
                e = max(e, b)
                f = max(f, c)
        return [d, e, f] == target
```

• SimplyLeet

```
def mergeTriplets(triplets, target):
    # Initialize candidate triplet with zeros
    candidate = [0, 0, 0]
    # Iterate over every triplet in the input list
    for trip in triplets:
        # Only consider triplets where each element is <= corresponding target element
        if trip[0] <= target[0] and trip[1] <= target[1] and trip[2] <= target[2]:
            # Update candidate by taking the element-wise maximum
            candidate[0] = max(candidate[0], trip[0])
            candidate[1] = max(candidate[1], trip[1])
```

```
        candidate[2] = max(candidate[2], trip[2])
    # If the candidate matches target, return True; otherwise, return False
    return candidate == target

# Example usage:
triplets = [[2,5,3],[1,8,4],[1,7,5]]
target = [2,7,5]
print(mergeTriplets(triplets, target)) # Output: True
```

◆ Quick Review

Key Insights

- Only consider triplets that are "valid" in the sense that none of its values exceed the corresponding target value.
- For each component of the target, there must be at least one valid triplet that provides that maximum value.
- By taking the element-wise maximum across all valid triplets, if we can recreate the target, then the target can be formed using a series of allowed operations.

Space & Time

Time Complexity: $O(n)$, where n is the number of triplets.

Space Complexity: $O(1)$ extra space.

Solution

The solution approach involves iterating over the list of triplets and only considering those triplets that do not exceed the target values in any position. For each valid triplet, update a candidate triplet by taking the element-wise maximum with the current candidate. Finally, check if the candidate exactly matches the target triplet. The algorithm uses simple array operations and avoids any extra space beyond a few variables.

Round 7 · Low · Easy

Round 7

Low Priority · Easy

3 questions · 2 patterns

1-D Dynamic Programming #746

Math & Geometry #66  #202

1-D Dynamic Programming

Round 7 · Low · Easy · 1 question

1-D Dynamic Programming

#746 Min Cost Climbing Stairs

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: NeetCode 150

Problem

You are given an integer array `cost` where `cost[i]` is the cost of *i*th step on a staircase. Once you pay the cost, you can either climb one or two steps.

You can either start from the step with index 0, or the step with index 1.

Return the minimum cost to reach the top of the floor.

Example 1:

```
Input: cost = [10,15,20]
Output: 15
Explanation: You will start at index 1.
- Pay 15 and climb two steps to reach the top.
The total cost is 15.
```

Example 2:

Input: cost = [1,100,1,1,1,100,1,1,100,1]

Output: 6

Explanation: You will start at index 0.

- Pay 1 and climb two steps to reach index 2.
- Pay 1 and climb two steps to reach index 4.
- Pay 1 and climb two steps to reach index 6.
- Pay 1 and climb one step to reach index 7.
- Pay 1 and climb two steps to reach index 9.

Constraints:

- $2 \leq \text{cost.length} \leq 1000$
- $0 \leq \text{cost}[i] \leq 999$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minCostClimbingStairs(self, cost: list[int]) -> int:
        cost.append(0)

        for i in range(2, len(cost)):
            cost[i] += min(cost[i - 1], cost[i - 2])

        return cost[-1]
```

• LeetDoocs

```
class Solution:
    def minCostClimbingStairs(self, cost: List[int]) -> int:
        @cache
        def dfs(i: int) -> int:
            if i >= len(cost):
                return 0
            return cost[i] + min(dfs(i + 1), dfs(i + 2))

        return min(dfs(0), dfs(1))
```

```
class Solution:
    def minCostClimbingStairs(self, cost: List[int]) -> int:
        n = len(cost)
        f = [0] * (n + 1)
        for i in range(2, n + 1):
            f[i] = min(f[i - 2] + cost[i - 2], f[i - 1] + cost[i - 1])
        return f[n]
```

```
class Solution:
    def minCostClimbingStairs(self, cost: List[int]) -> int:
        f = g = 0
        for i in range(2, len(cost) + 1):
            f, g = g, min(f + cost[i - 2], g + cost[i - 1])
        return g
```

• SimplyLeet

Python implementation with detailed comments

```
def minCostClimbingStairs(cost):
    n = len(cost) # number of steps
    # dp array to store minimum cost to reach each step
    dp = [0] * n
    dp[0] = cost[0] # cost to start at step 0
    dp[1] = cost[1] # cost to start at step 1

    # Compute minimum cost for each subsequent step

    for i in range(2, n):
        # cost to reach current step is current cost + min cost of the two
        # previous steps
        dp[i] = cost[i] + min(dp[i-1], dp[i-2])

    # The top can be reached from the last or second last step
    return min(dp[n-1], dp[n-2])

# Example usage:
cost = [10, 15, 20]
print(minCostClimbingStairs(cost))
```

◆ Quick Review

Key Insights

- Use Dynamic Programming to solve overlapping subproblems.
- The minimum cost to reach a step is the cost of that step plus the minimum cost of reaching one of the two previous steps.
- The final result is the minimum cost to reach the last or second last step since you can jump from either directly to the top.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$ (can be optimized to $O(1)$ by using only two variables)

Solution

We use a dynamic programming approach. Define a dp array where $dp[i]$ is the minimum cost required to reach step i . Initialize $dp[0]$ with $cost[0]$ and $dp[1]$ with $cost[1]$ because you can start at either step. For each subsequent step, the recurrence is $dp[i] = cost[i] + \min(dp[i-1], dp[i-2])$. Finally, the answer is the minimum of $dp[n-1]$ and $dp[n-2]$ since you can reach the top from either of these steps. This method efficiently

computes the answer in one pass through the array.

Math & Geometry

Round 7 · Low · Easy · 2 questions

Math & Geometry

#66 Plus One

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math

Lists: NeetCode 150

Problem

You are given a large integer represented as an integer array `digits`, where each `digits[i]` is the *i*th digit of the integer. The digits are ordered from most significant to least significant in left-to-right order. The large integer does not contain any leading 0's.

Increment the large integer by one and return the resulting array of digits.

Example 1:

Input: `digits = [1,2,3]`

Output: `[1,2,4]`

Explanation: The array represents the integer 123.

Incrementing by one gives $123 + 1 = 124$.

Thus, the result should be `[1,2,4]`.

Example 2:

Input: digits = [4,3,2,1]

Output: [4,3,2,2]

Explanation: The array represents the integer 4321.

Incrementing by one gives $4321 + 1 = 4322$.

Thus, the result should be [4,3,2,2].

Example 3:

Input: digits = [9]

Output: [1,0]

Explanation: The array represents the integer 9.

Incrementing by one gives $9 + 1 = 10$.

Thus, the result should be [1,0].

Constraints:

- $1 \leq \text{digits.length} \leq 100$
- $0 \leq \text{digits}[i] \leq 9$
- digits does not contain any leading 0's.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def plusOne(self, digits: list[int]) -> list[int]:
        for i, d in reversed(list(enumerate(digits))):
            if d < 9:
                digits[i] += 1
                return digits
            digits[i] = 0

        return [1] + digits
```

• LeetDoocs

```
class Solution:
    def plusOne(self, digits: List[int]) -> List[int]:
        n = len(digits)
        for i in range(n - 1, -1, -1):
            digits[i] += 1
            digits[i] %= 10
            if digits[i] != 0:
                return digits
        return [1] + digits
```

• SimplyLeet

```
# Function to add one to the array of digits
def plusOne(digits):
    # Process digits starting from the least significant digit
    for i in range(len(digits) - 1, -1, -1):
        # Add one to the current digit
        digits[i] += 1
        # Check if digit is less than 10, no carry needed
        if digits[i] < 10:
            return digits # No further carry, return the result
        # Set current digit to 0 because of carry over

        digits[i] = 0
    # If carry remains after processing all digits, add a new digit at start
    return [1] + digits

# Example usage
# print(plusOne([1, 2, 3])) # Output: [1, 2, 4]
```

◆ Quick Review

Key Insights

- The digits are stored from most significant to least significant.
- Start processing from the least significant digit (end of array) to handle potential carry.

- Carefully handle the case where all digits are 9, in which case an additional digit is needed.
- Use an in-place update approach to reduce extra space usage when possible.

Space & Time

Time Complexity: $O(n)$ in the worst-case, where n is the number of digits.

Space Complexity: $O(n)$ in the worst-case for the output array (when an extra digit is added).

Solution

We iterate from the end of the digits array towards the beginning, adding one to the current digit. If the digit becomes 10, we set it to 0 and propagate the carry to the next digit. If no carry is needed, we can return the result immediately. If we finish processing the array and still have a carry (e.g., the number was all 9s), we create a new array with one extra digit at the beginning set to 1.

Math & Geometry

#202 Happy Number

EAS

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Math · Two Pointers
Lists: NeetCode 150

Problem

Write an algorithm to determine if a number n is happy.

A happy number is a number defined by the following process:

- Starting with any positive integer, replace the number by the sum of the squares of its digits.
- Repeat the process until the number equals 1 (where it will stay), or it loops endlessly in a cycle which does not include 1.
- Those numbers for which this process ends in 1 are happy.

Return true if n is a happy number, and false if not.

Example 1:

Input: n = 19
 Output: true
 Explanation:
 12 + 92 = 82
 82 + 22 = 68
 62 + 82 = 100
 12 + 02 + 02 = 1

Example 2:

Input: n = 2
 Output: false

Constraints:

- $1 \leq n \leq 231 - 1$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isHappy(self, n: int) -> bool:
        def squaredSum(n: int) -> bool:
            summ = 0
            while n > 0:
                summ += pow(n % 10, 2)
                n //= 10
            return summ

        slow = squaredSum(n)

        fast = squaredSum(squaredSum(n))

        while slow != fast:
            slow = squaredSum(slow)
            fast = squaredSum(squaredSum(fast))

        return slow == 1
```

• LeetDoocs

```

class Solution:
    def isHappy(self, n: int) -> bool:
        vis = set()
        while n != 1 and n not in vis:
            vis.add(n)
            x = 0
            while n:
                n, v = divmod(n, 10)
                x += v * v
            n = x

        return n == 1

class Solution:
    def isHappy(self, n: int) -> bool:
        def next(x):
            y = 0
            while x:
                x, v = divmod(x, 10)
                y += v * v
            return y

        slow, fast = n, next(n)

        while slow != fast:
            slow, fast = next(slow), next(next(fast))
        return slow == 1

```

• SimplyLeet

```

# Function to compute the sum of squares of digits
def sum_of_squares(num):
    total = 0
    while num:
        digit = num % 10      # Extract the last digit
        total += digit * digit # Add its square to the total
        num //= 10           # Remove the last digit
    return total

def isHappy(n):

```

```
seen = set()                # Initialize a set to keep track of seen numbers
# Iterate until n becomes 1 or we detect a cycle
while n != 1 and n not in seen:
    seen.add(n)              # Add current n to the set
    n = sum_of_squares(n)    # Compute new n as the sum of squares of digits
return n == 1               # Return True if n is 1, else False

# Example usage:
print(isHappy(19))          # Expected output: True
print(isHappy(2))           # Expected output: False
```

◆ Quick Review

Key Insights

- The process involves repeatedly computing the sum of squares of digits.
- If the process reaches 1, the number is happy.
- If the computed values start repeating (cycle detection), the number is not happy.
- The cycle can be detected using a hash set to record previously seen numbers or using a two pointers (Floyd's cycle detection) approach.

Space & Time

Time Complexity: $O(1)$ – Although the process appears iterative, the sum of squares quickly reduces any number to a bounded range.

Space Complexity: $O(1)$ – The number of unique sums computed is bounded, resulting in constant space usage.

Solution

The solution involves computing the sum of the squares of a number's digits in each iteration. We use a hash set to track numbers that have already been seen. If a number repeats, it means we are in a cycle and the number is not happy. If the number becomes 1, it is a happy number. Data structures used include a hash set for cycle detection, and the algorithm uses iterative computation.

Round 8 · Low · Medium

Round 8

Low Priority · Medium

8 questions · 4 patterns

- ☐ ● 1-D Dynamic Programming #213 #647 ↗
- ☐ ● 2-D Dynamic Programming #97 #494 #518 ↗
- ☐ ● Bit Manipulation #371 ↗
- ☐ ● Math & Geometry #43 #2013 ↗

1-D Dynamic Programming

Round 8 · Low · Medium · 2 questions

1-D Dynamic Programming

□ #213 House Robber II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: NeetCode 150

Problem

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are arranged in a circle. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have a security system connected, and it will automatically contact the police if two adjacent houses were broken into on the same night.

Given an integer array `nums` representing the amount of money of each house, return the maximum amount of money you can rob tonight without alerting the police.

Example 1:

Input: nums = [2,3,2]

Output: 3

Explanation: You cannot rob house 1 (money = 2) and then rob house 3 (money = 2), because they are adjacent houses.

Example 2:

Input: nums = [1,2,3,1]

Output: 4

Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).
Total amount you can rob = 1 + 3 = 4.

Example 3:

Input: nums = [1,2,3]

Output: 3

Constraints:

- $1 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def rob(self, nums: list[int]) -> int:
        if not nums:
            return 0
        if len(nums) < 2:
            return nums[0]

        def rob(l: int, r: int) -> int:
            dp1 = 0
            dp2 = 0
```

```

    for i in range(l, r + 1):
        temp = dp1
        dp1 = max(dp1, dp2 + nums[i])
        dp2 = temp

    return dp1

return max(rob(0, len(nums) - 2),
           rob(1, len(nums) - 1))

```

• LeetCode

```

class Solution:
    def rob(self, nums: List[int]) -> int:
        def _rob(nums):
            f = g = 0
            for x in nums:
                f, g = max(f, g), f + x
            return max(f, g)

        if len(nums) == 1:
            return nums[0]

        return max(_rob(nums[1:]), _rob(nums[:-1]))

```

• SimplyLeet

```

# Python solution for House Robber II

def rob(nums):
    # Helper function to handle the linear House Robber problem.
    def rob_linear(houses):
        prev_max, curr_max = 0, 0
        for money in houses:
            temp = curr_max
            curr_max = max(curr_max, prev_max + money)
            prev_max = temp

```

```
        return curr_max

    n = len(nums)
    # When there's only one house, just return its value.
    if n == 1:
        return nums[0]

    # Rob excluding the first house and excluding the last house.
    return max(rob_linear(nums[1:]), rob_linear(nums[:-1]))

# Test example
print(rob([2, 3, 2])) # Expected output: 3
```

◆ Quick Review

Key Insights

- The houses are arranged in a circle, meaning the first and last houses are neighbors.
- Because of the circular arrangement, you cannot rob both the first and last houses.
- The problem can be reduced into two separate linear House Robber problems:
- One considering houses from index 1 to $n-1$ (exclude the first house).
- The other considering houses from index 0 to $n-2$ (exclude the last house).
- Use dynamic programming with constant space (using two variables) to compute the solution for a linear array of houses.

Space & Time

Time Complexity: $O(n)$, where n is the number of houses.

Space Complexity: $O(1)$, as only a constant amount of extra space is used.

Solution

To handle the circular dependency, we solve two linear versions of the problem:

- Exclude the first house and calculate the maximum rob amount for houses from index 1 to $n-1$.
- Exclude the last house and calculate the maximum rob amount for houses from index 0 to $n-2$.

Then, the final answer is the maximum of these two calculated values. We use dynamic programming by iterating through the list and at each step, calculating the maximum money that can be robbed up to that point with two variables.

1-D Dynamic Programming

□ #647 Palindromic Substrings

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String · Dynamic Programming
Lists: NeetCode 150

Problem

Given a string s , return the number of palindromic substrings in it.

A string is a palindrome when it reads the same backward as forward.

A substring is a contiguous sequence of characters within the string.

Example 1:

Input: $s = \text{"abc"}$
Output: 3
Explanation: Three palindromic strings: "a", "b", "c".

Example 2:

Input: $s = \text{"aaa"}$
Output: 6
Explanation: Six palindromic strings: "a", "a", "a", "aa", "aa", "aaa".

Constraints:

- $1 \leq s.length \leq 1000$
- s consists of lowercase English letters.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def countSubstrings(self, s: str) -> int:
        def extendPalindromes(l: int, r: int) -> int:
            count = 0

            while l >= 0 and r < len(s) and s[l] == s[r]:
                count += 1
                l -= 1
                r += 1

        return count

ans = 0

for i in range(len(s)):
    ans += extendPalindromes(i, i)
    ans += extendPalindromes(i, i + 1)

return ans
```

• LeetDoocs

```
class Solution:
    def countSubstrings(self, s: str) -> int:
        ans, n = 0, len(s)
        for k in range(n * 2 - 1):
            i, j = k // 2, (k + 1) // 2
            while ~i and j < n and s[i] == s[j]:
                ans += 1
                i, j = i - 1, j + 1
        return ans
```

```

class Solution:
    def countSubstrings(self, s: str) -> int:
        t = '^#' + '#'.join(s) + '#$'
        n = len(t)
        p = [0 for _ in range(n)]
        pos, maxRight = 0, 0
        ans = 0
        for i in range(1, n - 1):
            p[i] = min(maxRight - i, p[2 * pos - i]) if maxRight > i else 1
            while t[i - p[i]] == t[i + p[i]]:
                p[i] += 1
            if i + p[i] > maxRight:
                maxRight = i + p[i]
                pos = i
            ans += p[i] // 2
        return ans

```

• SimplyLeet

```

# Define a function to count palindromic substrings in a string
def countSubstrings(s: str) -> int:
    # Initialize the result variable to accumulate count of palindromic
    substrings
    result = 0

    # Helper function that expands around a center (left, right) and counts
    palindromes
    def expandAroundCenter(left: int, right: int) -> int:
        count = 0
        # Expand as long as indices are within bounds and characters at left
        and right are equal
        while left >= 0 and right < len(s) and s[left] == s[right]:

```

```
        # Found a palindrome, so increment count
        count += 1
        left -= 1 # Move left pointer outward
        right += 1 # Move right pointer outward
    return count

# Iterate over each character in the string
for i in range(len(s)):
    # Count odd-length palindromes (center is i)
    result += expandAroundCenter(i, i)

    # Count even-length palindromes (center is between i and i+1)
    result += expandAroundCenter(i, i + 1)

return result

# Example usage:
# result = countSubstrings("aaa")
# print(result)
```

◆ Quick Review

Key Insights

- Every individual character is a palindrome.
- Palindromes can be expanded from a center point; centers can be single characters (odd length) or between two characters (even length).
- The "expand around center" technique efficiently counts palindromes in $O(n^2)$ time.
- Dynamic programming could also be used but requires extra space and is more complex to implement.

Space & Time

Time Complexity: $O(n^2)$ where n is the length of the string.

Space Complexity: $O(1)$ if using the expand-around-center method (ignoring recursion/stack overhead).

Solution

The solution employs the expand-around-center approach:

- Iterate over each index of the string considering it as the center of a palindrome.
- For each center, expand in both directions (left and right) as long as the characters are the same and within bounds.
- Count every valid palindrome found during the expansion.
- Repeat for both odd-length centers (a single index) and even-length centers (between two indices).

This method efficiently checks all potential palindromic substrings without extra space for dynamic programming tables.

2-D Dynamic Programming

Round 8 · Low · Medium · 3 questions

2-D Dynamic Programming

□ #97 Interleaving String

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: NeetCode 150

Problem

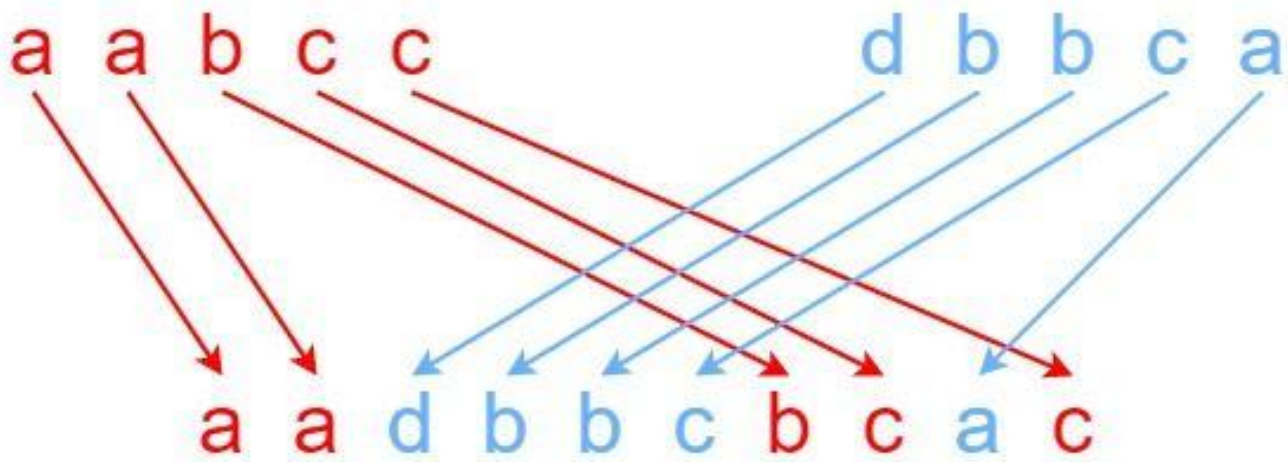
Given strings s_1 , s_2 , and s_3 , find whether s_3 is formed by an interleaving of s_1 and s_2 .

An interleaving of two strings s and t is a configuration where s and t are divided into n and m substrings respectively, such that:

- $s = s_1 + s_2 + \dots + s_n$
- $t = t_1 + t_2 + \dots + t_m$
- $|n - m| \leq 1$
- The interleaving is $s_1 + t_1 + s_2 + t_2 + s_3 + t_3 + \dots$ or $t_1 + s_1 + t_2 + s_2 + t_3 + s_3 + \dots$

Note: $a + b$ is the concatenation of strings a and b .

Example 1:



Input: s1 = "aabcc", s2 = "dbbca", s3 = "aadbcbcbcac"

Output: true

Explanation: One way to obtain s3 is:

Split s1 into s1 = "aa" + "bc" + "c", and s2 into s2 = "dbbc" + "a".

Interleaving the two splits, we get "aa" + "dbbc" + "bc" + "a" + "c" = "aadbcbcbcac".

Since s3 can be obtained by interleaving s1 and s2, we return true.

Example 2:

Input: s1 = "aabcc", s2 = "dbbca", s3 = "aadbabbaccc"

Output: false

Explanation: Notice how it is impossible to interleave s2 with any other string to obtain s3.

Example 3:

Input: s1 = "", s2 = "", s3 = ""

Output: true

Constraints:

- $0 \leq s1.length, s2.length \leq 100$
- $0 \leq s3.length \leq 200$
- s1, s2, and s3 consist of lowercase English letters.

Follow up: Could you solve it using only $O(s2.length)$ additional memory space?

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
        m = len(s1)
        n = len(s2)
        if m + n != len(s3):
            return False

        # dp[i][j] := true if s3[0..i + j) is formed by the interleaving of
        # s1[0..i) and s2[0..j)
        dp = [[False] * (n + 1) for _ in range(m + 1)]

        dp[0][0] = True

        for i in range(1, m + 1):
            dp[i][0] = dp[i - 1][0] and s1[i - 1] == s3[i - 1]

        for j in range(1, n + 1):
            dp[0][j] = dp[0][j - 1] and s2[j - 1] == s3[j - 1]

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                dp[i][j] = (dp[i - 1][j] and s1[i - 1] == s3[i + j - 1] or
                           dp[i][j - 1] and s2[j - 1] == s3[i + j - 1])

        return dp[m][n]
```

• LeetDoocs

```

class Solution:
    def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
        @cache
        def dfs(i: int, j: int) -> bool:
            if i >= m and j >= n:
                return True
            k = i + j
            if i < m and s1[i] == s3[k] and dfs(i + 1, j):
                return True
            if j < n and s2[j] == s3[k] and dfs(i, j + 1):
                return True
            return False

        m, n = len(s1), len(s2)
        if m + n != len(s3):
            return False
        return dfs(0, 0)

```

```

class Solution:
    def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
        m, n = len(s1), len(s2)
        if m + n != len(s3):
            return False
        f = [[False] * (n + 1) for _ in range(m + 1)]
        f[0][0] = True
        for i in range(m + 1):
            for j in range(n + 1):
                k = i + j - 1
                if i and s1[i - 1] == s3[k]:
                    f[i][j] = f[i - 1][j]
                if j and s2[j - 1] == s3[k]:
                    f[i][j] |= f[i][j - 1]
        return f[m][n]

```

• SimplyLeet

```
def isInterleave(s1, s2, s3):
    # Check if the combined lengths of s1 and s2 match s3
    if len(s1) + len(s2) != len(s3):
        return False
    # Initialize dp table with dimensions (len(s1)+1) x (len(s2)+1)
    dp = [[False] * (len(s2) + 1) for _ in range(len(s1) + 1)]
    dp[0][0] = True # Base case: empty strings form an empty string
    # Fill first row using characters from s2 only
    for j in range(1, len(s2) + 1):
        dp[0][j] = dp[0][j - 1] and s2[j - 1] == s3[j - 1]

    # Fill first column using characters from s1 only
    for i in range(1, len(s1) + 1):
        dp[i][0] = dp[i - 1][0] and s1[i - 1] == s3[i - 1]
    # Fill the rest of the dp table
    for i in range(1, len(s1) + 1):
        for j in range(1, len(s2) + 1):
            dp[i][j] = (dp[i - 1][j] and s1[i - 1] == s3[i + j - 1]) or \
                (dp[i][j - 1] and s2[j - 1] == s3[i + j - 1])
    return dp[len(s1)][len(s2)]

# Example usage:
print(isInterleave("aabcc", "dbbca", "aadbbcbcac")) # Expected output: True
print(isInterleave("aabcc", "dbbca", "aadbbbacc")) # Expected output: False
```

◆ Quick Review

Key Insights

- The problem asks if $s3$ can be constructed by preserving the order of characters in $s1$ and $s2$.
- A dynamic programming approach is effective by defining $dp[i][j]$ to represent whether $s3[0 \dots i+j-1]$ is an interleaving of $s1[0 \dots i-1]$ and $s2[0 \dots j-1]$.

- The solution must check that the total lengths match; otherwise, it is immediately false.
- Transitions are based on whether the current character in s3 matches the next char in s1 or s2.
- Space optimization to $O(s2.length)$ is possible by reusing a 1D DP array.

Space & Time

Time Complexity: $O(n * m)$ where n = length of s1 and m = length of s2.

Space Complexity: $O(n * m)$ with the basic DP solution. (Can be optimized to $O(m)$ with further space reduction.)

Solution

We use a 2D dynamic programming table where $dp[i][j]$ is true if $s3[0 \dots i+j-1]$ can be formed by interleaving $s1[0 \dots i-1]$ and $s2[0 \dots j-1]$. First, we check if the lengths of s1 and s2 add up to s3's length; if not, return false. Then, we initialize $dp[0][0]$ to true and fill in the first row and column based on whether the characters in s2 or s1 correspond to s3. For the rest of the table, $dp[i][j]$ is true if either the

previous state $dp[i-1][j]$ is true and $s1[i-1]$ matches $s3[i+j-1]$, or $dp[i][j-1]$ is true and $s2[j-1]$ matches $s3[i+j-1]$. The answer is found in $dp[s1.length][s2.length]$.

2-D Dynamic Programming

□ #494 Target Sum

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Backtracking
Lists: NeetCode 150

Problem

You are given an integer array `nums` and an integer `target`.

You want to build an expression out of `nums` by adding one of the symbols '+' and '-' before each integer in `nums` and then concatenate all the integers.

- For example, if `nums = [2, 1]`, you can add a '+' before 2 and a '-' before 1 and concatenate them to build the expression "+2-1".

Return the number of different expressions that you can build, which evaluates to `target`.

Example 1:

Input: nums = [1,1,1,1,1], target = 3

Output: 5

Explanation: There are 5 ways to assign symbols to make the sum of nums be target 3.

-1 + 1 + 1 + 1 + 1 = 3

+1 - 1 + 1 + 1 + 1 = 3

+1 + 1 - 1 + 1 + 1 = 3

+1 + 1 + 1 - 1 + 1 = 3

+1 + 1 + 1 + 1 - 1 = 3

Example 2:

Input: nums = [1], target = 1

Output: 1

Constraints:

- $1 \leq \text{nums.length} \leq 20$
- $0 \leq \text{nums}[i] \leq 1000$
- $0 \leq \text{sum}(\text{nums}[i]) \leq 1000$
- $-1000 \leq \text{target} \leq 1000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findTargetSumWays(self, nums: list[int], target: int) -> int:
        summ = sum(nums)
        if summ < abs(target) or (summ + target) % 2 == 1:
            return 0

        def knapsack(target: int) -> int:
            # dp[i] := the number of ways to sum to i by nums so far
            dp = [1] + [0] * summ
```

```

    for num in nums:
        for j in range(summ, num - 1, -1):
            dp[j] += dp[j - num]

    return dp[target]

return knapsack((summ + target) // 2)

```

• LeetCode

```

class Solution:
    def findTargetSumWays(self, nums: List[int], target: int) -> int:
        s = sum(nums)
        if s < target or (s - target) % 2:
            return 0
        m, n = len(nums), (s - target) // 2
        f = [[0] * (n + 1) for _ in range(m + 1)]
        f[0][0] = 1
        for i, x in enumerate(nums, 1):
            for j in range(n + 1):
                f[i][j] = f[i - 1][j]
                if j >= x:
                    f[i][j] += f[i - 1][j - x]
        return f[m][n]

```

• SimplyLeet

```

# Define a function to compute the number of ways to reach the target sum.
def findTargetSumWays(nums, target):
    # Calculate the total sum of all numbers in the list.
    total_sum = sum(nums)
    # If the target is greater than total_sum, or (target + total_sum) is odd,
    no solution exists.
    if target > total_sum or (target + total_sum) % 2 == 1:
        return 0

    # Compute the required subset sum.
    subset_sum = (target + total_sum) // 2

```

```
# Initialize the dp array with zeros, with dp[0] set to 1.
dp = [0] * (subset_sum + 1)
dp[0] = 1

# Update the dp array for each number in nums.
for num in nums:
    # Iterate backwards to ensure each number is only counted once.
    for s in range(subset_sum, num - 1, -1):
        dp[s] += dp[s - num]
return dp[subset_sum]
```

```
# Example usage:
print(findTargetSumWays([1,1,1,1,1], 3)) # Expected output: 5
```

◆ Quick Review

Key Insights

- The problem can be converted into a subset sum problem by considering two groups: numbers with a '+' sign and numbers with a '-' sign.
- Let the sum of numbers with a '+' sign be $\text{sum}(P)$ and with a '-' sign be $\text{sum}(N)$. Then the problem reduces to solving:
 - $\text{sum}(P) - \text{sum}(N) = \text{target}$
 - and $\text{sum}(P) + \text{sum}(N) = \text{totalSum}$, where totalSum is the sum of all numbers in nums .
- From these equations, we derive $\text{sum}(P) = (\text{target} + \text{totalSum}) / 2$.

- We only proceed if $(\text{target} + \text{totalSum})$ is even and not negative; otherwise, there are no valid expressions.
- Dynamic programming is used to count the number of subsets that sum to $\text{sum}(P)$.

Space & Time

Time Complexity: $O(n * \text{subsetSum})$, where n is the length of the `nums` array and $\text{subsetSum} = (\text{target} + \text{totalSum}) / 2$.

Space Complexity: $O(\text{subsetSum})$, as we use a one-dimensional DP array.

Solution

The solution converts the problem into a subset sum DP problem. First, compute the total sum of the array. If $(\text{target} + \text{totalSum})$ is not even or target is greater than totalSum , return 0 since no valid configuration exists. Otherwise, compute the required subset sum (subsetSum) as $(\text{target} + \text{totalSum}) / 2$ and use a dynamic programming array to count the number of ways to reach subsetSum . The DP approach iterates through each number in the array, updating the counts using a reverse iteration to ensure each number is only used

once.

2-D Dynamic Programming

□ #518 Coin Change II

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: NeetCode 150

Problem

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money.

Return the number of combinations that make up that amount. If that amount of money cannot be made up by any combination of the coins, return 0.

You may assume that you have an infinite number of each kind of coin.

The answer is guaranteed to fit into a signed 32-bit integer.

Example 1:

Input: amount = 5, coins = [1,2,5]
 Output: 4
 Explanation: there are four ways to make up the amount:
 5=5
 5=2+2+1
 5=2+1+1+1
 5=1+1+1+1+1

Example 2:

Input: amount = 3, coins = [2]
 Output: 0
 Explanation: the amount of 3 cannot be made up just with coins of 2.

Example 3:

Input: amount = 10, coins = [10]
 Output: 1

Constraints:

- $1 \leq \text{coins.length} \leq 300$
- $1 \leq \text{coins}[i] \leq 5000$
- All the values of coins are unique.
- $0 \leq \text{amount} \leq 5000$

Community Solutions (Python)

• WalkCC

```
class Solution:
    def change(self, amount: int, coins: list[int]) -> int:
        dp = [1] + [0] * amount

        for coin in coins:
            for i in range(coin, amount + 1):
                dp[i] += dp[i - coin]

        return dp[amount]
```

• LeetCode

```
class Solution:
    def change(self, amount: int, coins: List[int]) -> int:
        m, n = len(coins), amount
        f = [[0] * (n + 1) for _ in range(m + 1)]
        f[0][0] = 1
        for i, x in enumerate(coins, 1):
            for j in range(n + 1):
                f[i][j] = f[i - 1][j]
                if j >= x:
                    f[i][j] += f[i][j - x]

        return f[m][n]
```

```
class Solution:
    def change(self, amount: int, coins: List[int]) -> int:
        n = amount
        f = [1] + [0] * n
        for x in coins:
            for j in range(x, n + 1):
                f[j] += f[j - x]
        return f[n]
```

• SimplyLeet

```
# Python solution for Coin Change II
def change(amount, coins):
    # Initialize dp array where dp[j] represents the number of ways to form
    # amount j
    dp = [0] * (amount + 1)
    dp[0] = 1 # One way to achieve amount 0 (use no coins)

    # Process each coin
    for coin in coins:
        # Update dp for all amounts from coin to target amount
        for j in range(coin, amount + 1):
            dp[j] += dp[j - coin] # Accumulate ways by using coin
    return dp[amount]

# Example usage:
print(change(5, [1, 2, 5])) # Output: 4
```

◆ Quick Review

Key Insights

- Use dynamic programming to build up the number of ways to form every amount up to the target.
- A one-dimensional dp array suffices, where $dp[j]$ represents the number of ways to form sum j .
- When processing each coin, iterate over amounts from the coin value to the target, updating dp.
- Order of iteration ensures that combinations, not permutations, are counted.

Space & Time

Time Complexity: $O(n * \text{amount})$, where n is the number of coins.

Space Complexity: $O(\text{amount})$, due to the dp array used for computations.

Solution

We solve the problem by using dynamic programming with a one-dimensional dp array. The dp array is sized as $\text{amount} + 1$

with $dp[0]$ initialized to 1, representing the single way to achieve the amount of 0 (by choosing no coins). For each coin in the coins array, we update every $dp[j]$ for j from the coin's value up to the target amount: $dp[j]$ is increased by $dp[j - \text{coin}]$. This ensures we consider each coin only once per combination, hence avoiding duplicate counts of the same combination in different orders.

Bit Manipulation

Round 8 · Low · Medium · 1 question

Bit Manipulation

#371 Sum of Two Integers

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Bit Manipulation

Lists: NeetCode 150

Problem

Given two integers a and b , return the sum of the two integers without using the operators $+$ and $-$.

Example 1:

Input: $a = 1, b = 2$
Output: 3

Example 2:

Input: $a = 2, b = 3$
Output: 5

Constraints:

- $-1000 \leq a, b \leq 1000$

Community Solutions (Python)

- WalkCC

```
class Solution:
    def getSum(self, a: int, b: int) -> int:
        mask = 0xFFFFFFFF
        MAX = 2000

        while b != 0:
            a, b = (a ^ b) & mask, ((a & b) << 1) & mask

        return a if a < MAX else ~(a ^ mask)
```

• LeetDoocs

```
class Solution:
    def getSum(self, a: int, b: int) -> int:
        a, b = a & 0xFFFFFFFF, b & 0xFFFFFFFF
        while b:
            carry = ((a & b) << 1) & 0xFFFFFFFF
            a, b = a ^ b, carry
        return a if a < 0x80000000 else ~(a ^ 0xFFFFFFFF)
```

• SimplyLeet

```
# Python implementation to add two integers without using + or -
def getSum(a, b):
    # Continue till there is no carry
    while b != 0:
        # XOR gets the sum excluding carries
        sum_without_carry = a ^ b
        # AND finds carries, left shift to apply them to the next bit
        carry = (a & b) << 1
        # Update a with sum without carry and b with carry
        a = sum_without_carry

        b = carry
    return a

# Example usage:
result = getSum(1, 2)
print("Sum:", result) # Output: Sum: 3
```

◆ Quick Review

Key Insights

- Use bitwise XOR (^) to perform addition without carrying.
- Use bitwise AND (&) followed by a left shift (<<) to calculate the carry.
- Repeat the process until there is no carry.
- This method simulates the addition process using binary arithmetic.

Space & Time

Time Complexity: $O(1)$ – The loop runs a fixed number of times based on the number of bits.

Space Complexity: $O(1)$ – Only constant extra space is used.

Solution

The solution uses bit manipulation to add two numbers without using the + or – operators.

- Use XOR (^) on a and b to add the bits where there is no carry.
- Use AND (&) to find the positions where both a and b have a 1 (i.e., potential carry). Then shift the result left by one to add in the next higher bit.
- Set a to the XOR result and b to the shifted carry.
- Repeat steps 1–3 until there is no carry (b equals 0).
-

Return a which holds the final sum. This approach leverages the bitwise operations to effectively simulate binary addition.

Math & Geometry

Round 8 · Low · Medium · 2 questions

Math & Geometry

#43 Multiply Strings

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · String · Simulation

Lists: NeetCode 150

Problem

Given two non-negative integers `num1` and `num2` represented as strings, return the product of `num1` and `num2`, also represented as a string.

Note: You must not use any built-in BigInteger library or convert the inputs to integer directly.

Example 1:

Input: `num1 = "2"`, `num2 = "3"`
Output: `"6"`

Example 2:

Input: `num1 = "123"`, `num2 = "456"`
Output: `"56088"`

Constraints:

- $1 \leq \text{num1.length}, \text{num2.length} \leq 200$
- `num1` and `num2` consist of digits only.
- Both `num1` and `num2` do not contain any leading zero, except the number 0 itself.

Community Solutions (Python)

• WalkCC

```
class Solution:
    def multiply(self, num1: str, num2: str) -> str:
        s = [0] * (len(num1) + len(num2))

        for i in reversed(range(len(num1))):
            for j in reversed(range(len(num2))):
                mult = int(num1[i]) * int(num2[j])
                summ = mult + s[i + j + 1]
                s[i + j] += summ // 10
                s[i + j + 1] = summ % 10

        for i, c in enumerate(s):
            if c != 0:
                break

        return ''.join(map(str, s[i:]))
```

• LeetDoocs

```
class Solution:
    def multiply(self, num1: str, num2: str) -> str:
        if num1 == "0" or num2 == "0":
            return "0"

        m, n = len(num1), len(num2)
        arr = [0] * (m + n)
        for i in range(m - 1, -1, -1):
            a = int(num1[i])
            for j in range(n - 1, -1, -1):
                b = int(num2[j])

                arr[i + j + 1] += a * b
            for i in range(m + n - 1, 0, -1):
                arr[i - 1] += arr[i] // 10
                arr[i] %= 10
            i = 0 if arr[0] else 1
        return "".join(str(x) for x in arr[i:])
```

• SimplyLeet

```
# Python implementation with line-by-line comments

def multiply(num1, num2):
    # If either string is "0", the product is "0".
    if num1 == "0" or num2 == "0":
        return "0"

    n, m = len(num1), len(num2)
    # Initialize an array to store the product results.
    result = [0] * (n + m)

    # Multiply each digit of num1 with each digit of num2.
    for i in range(n - 1, -1, -1):
        for j in range(m - 1, -1, -1):
            # Multiply current digits.
            product = int(num1[i]) * int(num2[j])
            # Position in the result array where the product should be added.
            pos = i + j + 1
            total = product + result[pos]
            result[pos] = total % 10 # Place the digit at the current index.

    result[pos - 1] += total // 10 # Carry over the remaining value.

    # Convert the result array into a string, skipping any leading zero.
    result_str = ''.join(map(str, result)).lstrip('0')
    return result_str if result_str else "0"

# Example usage:
print(multiply("123", "456")) # Expected output: "56088"
```

◆ Quick Review

Key Insights

- Simulate the multiplication process similar to hand multiplication.

- Use an array to store intermediate results where the index correlation is based on the position of digits.
- Each multiplication may contribute a value that needs to be managed with a carry over in the result array.
- Handle the special case when either input is "0".

Space & Time

Time Complexity: $O(n * m)$, where n and m are the lengths of `num1` and `num2` respectively.

Space Complexity: $O(n + m)$, used for storing the intermediate multiplication results.

Solution

The solution multiplies each digit of `num1` with each digit of `num2` by simulating the grade-school multiplication algorithm. An array is used to store the intermediate sums at positions that correspond to the sum of the indices from `num1` and `num2`. For two digits multiplied together, the product is added to the corresponding index in the result array, and any carry is added to the adjacent left cell. Finally, the array is converted back to a string

while taking care to remove any leading zeros.

Data structures used:

- Array for intermediate multiplication results.**
- Two nested loops to multiply each digit pair.**

Algorithmic steps:

- Initialize an array of size (length of num1 + length of num2) with all zeros.**
- Loop through digits of num1 and num2 in reverse order.**
- Multiply the two digits, add the product to the corresponding position in the result array (taking into account any previous values).**
- Manage the carry for each index.**
- Convert the result array to a string, skipping any leading zeros.**
- Return the final product string.**

Math & Geometry

#2013 Detect Squares

MED

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Design · Counting · Data
Stream

Lists: NeetCode 150

Problem

You are given a stream of points on the X-Y plane. Design an algorithm that:

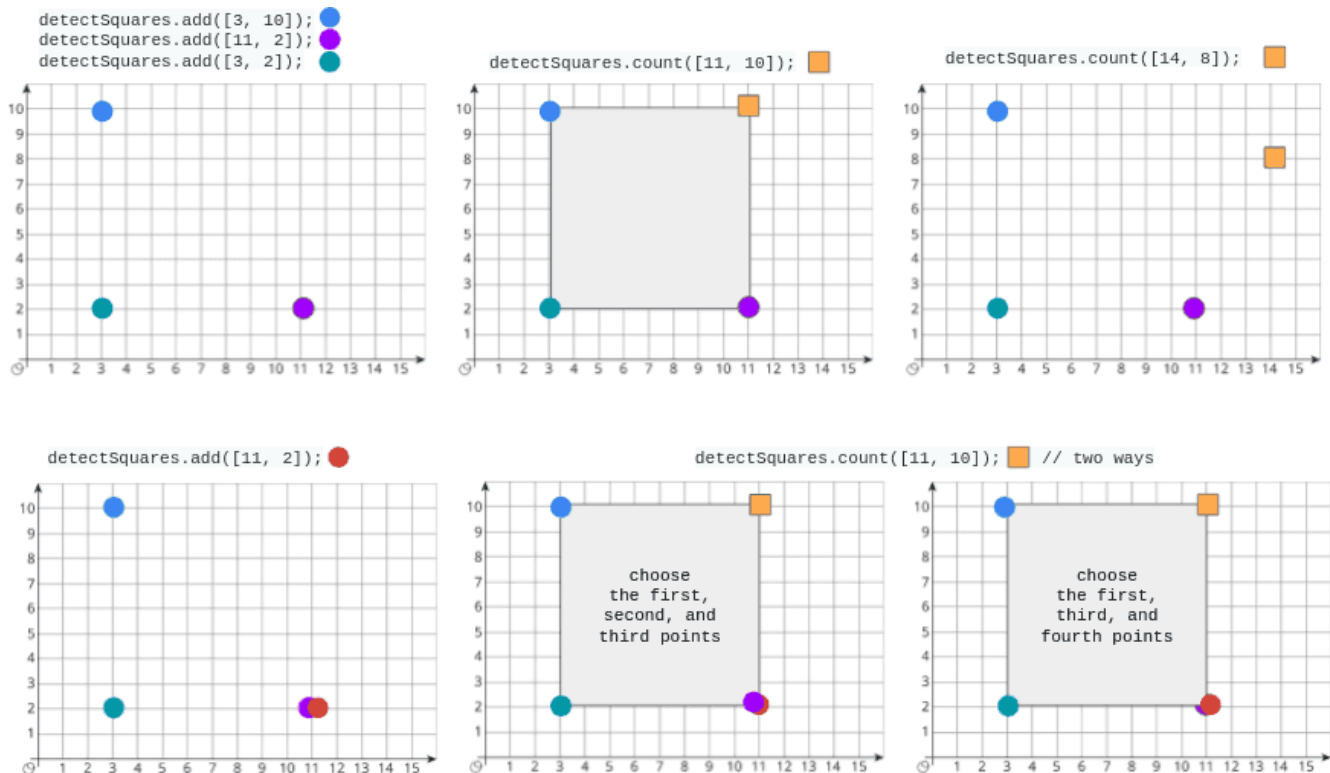
- Adds new points from the stream into a data structure. Duplicate points are allowed and should be treated as different points.
- Given a query point, counts the number of ways to choose three points from the data structure such that the three points and the query point form an axis-aligned square with positive area.

An axis-aligned square is a square whose edges are all the same length and are either parallel or perpendicular to the x-axis and y-axis.

Implement the DetectSquares class:

- **DetectSquares()** Initializes the object with an empty data structure.
- **void add(int[] point)** Adds a new point $\text{point} = [x, y]$ to the data structure.
- **int count(int[] point)** Counts the number of ways to form axis-aligned squares with point $\text{point} = [x, y]$ as described above.

Example 1:



Input

```
["DetectSquares", "add", "add", "add", "count", "count", "add", "count"]
[[], [[3, 10]], [[11, 2]], [[3, 2]], [[11, 10]], [[14, 8]], [[11, 2]], [[11, 10]]]
```

Output

```
[null, null, null, null, 1, 0, null, 2]
```

Explanation

```
DetectSquares detectSquares = new DetectSquares();
```

Constraints:

- `point.length == 2`
- `0 <= x, y <= 1000`
- At most 3000 calls in total will be made to add and count.

Community Solutions (Python)

• WalkCC

```
class DetectSquares:
    def __init__(self):
        self.pointCount = collections.Counter()

    def add(self, point: list[int]) -> None:
        self.pointCount[tuple(point)] += 1

    def count(self, point: list[int]) -> int:
        x1, y1 = point
        ans = 0

        for (x3, y3), c in self.pointCount.items():
            if x1 != x3 and abs(x1 - x3) == abs(y1 - y3):
                ans += c * self.pointCount[(x1, y3)] * self.pointCount[(x3, y1)]
        return ans
```

• LeetDoocs

```
class DetectSquares:
    def __init__(self):
        self.cnt = defaultdict(Counter)

    def add(self, point: List[int]) -> None:
        x, y = point
        self.cnt[x][y] += 1

    def count(self, point: List[int]) -> int:
        x1, y1 = point
```

```

        if x1 not in self.cnt:
            return 0
        ans = 0
        for x2 in self.cnt.keys():
            if x2 != x1:
                d = x2 - x1
                ans += self.cnt[x2][y1] * self.cnt[x1][y1 + d] * self.cnt[x2][
y1 + d]
                ans += self.cnt[x2][y1] * self.cnt[x1][y1 - d] * self.cnt[x2][
y1 - d]
        return ans

```

```

# Your DetectSquares object will be instantiated and called as such:
# obj = DetectSquares()
# obj.add(point)
# param_2 = obj.count(point)

```

• SimplyLeet

```

# Python implementation of DetectSquares
from collections import Counter

class DetectSquares:
    def __init__(self):
        # Use a counter to store the frequency of each point (x, y)
        self.point_counts = Counter()

    def add(self, point):
        x, y = point[0], point[1]

        # Increment the count for the point (x, y)
        self.point_counts[(x, y)] += 1

    def count(self, point):
        x, y = point[0], point[1]
        total_squares = 0
        # Iterate through all points in our data structure
        for (px, py), count in self.point_counts.items():
            # Only consider points that share the same y-coordinate with the
            query point
            if py == y and px != x:

```

```

        # Calculate the potential square's side length
        d = px - x
        # Check for a square above the query point
        total_squares += count * self.point_counts.get((x, y + d), 0)
    * self.point_counts.get((px, y + d), 0)
        # Check for a square below the query point
        total_squares += count * self.point_counts.get((x, y - d), 0)
    * self.point_counts.get((px, y - d), 0)
    return total_squares

# Example Usage:
# detectSquares = DetectSquares()

# detectSquares.add([3, 10])
# detectSquares.add([11, 2])
# detectSquares.add([3, 2])
# print(detectSquares.count([11, 10])) # Output: 1

```

◆ Quick Review

Key Insights

- Leverage the fact that an axis-aligned square has vertical and horizontal edges.
- For a query point (x, y) , iterate over all points that share the same y -axis. For each of these points (px, y) (with $px \neq x$), determine the potential square's side length, $d = |px - x|$.
- Check for the existence of the two other points $(x, y+d)$ and $(px, y+d)$ (or $(x, y-d)$ and $(px, y-d)$) that would complete the square.
- Use a hash map (or dictionary) to store each point along with its count since duplicate

points are allowed.

Space & Time

Time Complexity:

- The add operation is $O(1)$.
- The count operation is $O(n)$ in the worst-case scenario, where n is the number of distinct points with the same y -coordinate as the query point.

Space Complexity:

- $O(m)$, where m is the number of added points, since we store counts for each point.

Solution

We implement the `DetectSquares` class by maintaining a hash map (or dictionary) to store the frequency count for each point using its (x, y) coordinates as the key. In the `add(point)` method, we simply increment the count for the given point. For the `count(point)` method, we iterate through all points that share the same y -coordinate as the query point. For every such point (px, y) (with $px \neq x$), we calculate the potential side length of a square ($d = px - x$). Then, we check for two possible squares: one above the query point

and one below it. The count is increased by the number of times these needed points exist multiplied by the count of the current candidate point. This method efficiently counts all possible squares that can be formed with the query point.